THESIS

JAVASCRIPT VULNERABILITY DETECTION USING TABM ARCHITECTURES

RAMA ANDHIKA PRATAMA
23/517333/PA/22175

STUDY PROGRAM COMPUTER SCIENCE
DEPARTMENT COMPUTER SCIENCE AND ELECTRONICS
FACULTY OF MATHEMATICS AND NATURAL SCIENCES
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

SKRIPSI

DETEKSI KERENTANAN DALAM JAVASCRIPT MENGGUNAKAN ARSITEKTUR TABM

JAVASCRIPT VULNERABILITY DETECTION USING TABM ARCHITECTURES

Diajukan untuk memenuhi salah satu syarat memperoleh derajat
Bachelor of Computer Science



RAMA ANDHIKA PRATAMA
23/517333/PA/22175

PROGRAM STUDI COMPUTER SCIENCE
DEPERTEMEN COMPUTER SCIENCE AND ELECTRONICS
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

UNDERGRADUATE THESIS

JAVASCRIPT VULNERABILITY DETECTION USING TABM ARCHITECTURES

DETEKSI KERENTANAN DALAM JAVASCRIPT MENGGUNAKAN ARSITEKTUR TABM

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Computer Science



RAMA ANDHIKA PRATAMA
23/517333/PA/22175

STUDY PROGRAM COMPUTER SCIENCE
DEPARTMENT COMPUTER SCIENCE AND ELECTRONICS
FACULTY OF MATHEMATICS AND NATURAL SCIENCES
GADJAH MADA UNIVERSITY
YOGYAKARTA

2026

APPROVAL PAGE

SKRIPSI

DETEKSI KERENTANAN DALAM JAVASCRIPT MENGGUNAKAN ARSITEKTUR TABM

Telah dipersiapkan dan disusun oleh

RAMA ANDHIKA PRATAMA
23/517333/PA/22175

Telah dipertahankan di depan Tim Penguji
pada tanggal 2 November 2026

Susunan Tim Penguji

Arif Nurwidyantoro, S.Kom., M.Cs., Ph.D. Pembimbing	NAMA DOSEN PENGUJI 1 Ketua Penguji
	NAMA DOSEN PENGUJI 2 Anggota Penguji

DECLARATION

Dengan ini saya menyatakan bahwa dalam Skripsi ini tidak terdapat karya yang pernah diajukan untuk memperoleh gelar kesarjanaan di suatu Perguruan Tinggi, dan sepanjang pengetahuan saya juga tidak terdapat karya atau pendapat yang ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis diacu dalam naskah ini dan disebutkan dalam daftar pustaka.

Yogyakarta, 2 November 2026

Rama Andhika Pratama

PREFACE

Isi Prakata di sini...

CONTENTS

Title Page ii

Title Page iii

Approval Page iv

Declaration v

Motto vi

Dedication vii

PREFACE viii

PREFACE viii

INTISARI xiv

ABSTRACT xv

I INTRODUCTION 1

1.1 Background 1
1.2 Problem Formulation 3
1.3 Research Limitations 3
1.4 Research Objectives 4
1.5 Research Benefits 4

II LITERATURE REVIEW 6

2.1 Tabular Software Metrics and Vulnerability Detection 6
2.2 Comparative Analysis of Previous Vulnerability Detection Models . . 7
2.3 Identified Research Gap and Significance of the Study 10

III RESEARCH METHODOLOGY 12

3.1 General Description 12
3.2 Data Acquisition 13
3.2.1 Ferenc et al. (2019) Dataset 13

3.2.2 Viszkok et al. (2021) Dataset 14
3.3 Data Preprocessing 15
3.4 Data Splitting 16
3.5 Class Imbalance Analysis and Handling 17
3.6 Model Implementation 18
3.6.1 TabM (Native Tabular Deep Learning) 18
3.6.2 FT-Transformer 21
3.6.3 Gradient Boosted Decision Trees (GDBT) 22
3.7 Hyperparameter Tuning 24
3.8 Model Evaluation 26
3.8.1 Performance Mentrics 27
3.8.2 Efficiency Metrics 27
3.9 Hardware and Software Environment 28

IV IMPLEMENTATION **29**
4.1 Data Pipeline Implementation 29
4.1.1 Data Acquisition 29
4.1.2 Preprocessing 29
4.1.3 Data Split 31
4.1.4 Class Imbalance Handling 32
4.2 Model & Training Pipeline Implementation 34
4.2.1 XGBoost 34
4.2.2 TabM 35
4.2.3 FT-Transformer 36
4.3 Tuning & Training-loop Implementation 37
4.3.1 XGBoost Search 37
4.3.2 TabM and FT-Transformer Search 37
4.3.3 Final Retrain 38
4.4 Evaluation and Comparison Implementation 38
4.4.1 Performance Fields 39
4.4.2 Efficiency Fields including Training Time and Testing Time . 39
4.4.3 How the Comparison Table is Generated 40

V RESULTS AND DISCUSSION **42**
5.1 Experimental Design 42
5.2 Experimental Results 42

5.2.1 Best Hyperparameters 42
5.2.2 Performance Results 44
5.2.3 Efficiency Results 46
5.3 Discussion 47
5.3.1 Predictive Performance 47
5.3.2 Computational Efficiency 48
5.4 Comparison With Prior Works 49
5.5 Limitations of the Finding 51

A ATTACHMENT **56**
1.1 Ferenc Dataset Summary Statistic Table 56
1.2 Viszkok Dataset Summary Statistic Table 57

LIST OF TABLES

2.1 Comparison of Recent Vulnerability Prediction Methodologies 8

3.1 Data Split Configuration for Ferenc's Dataset 16
3.2 Data Split Configuration for Viszkok's Dataset 16
3.3 Hyperparameter Tuning Configuration for All Three Architectures . . 25

4.1 Column removals details for Ferenc et al. (2019) dataset 30
4.2 Column removals details for Viszkok et al. (2021) dataset 30
4.3 Realized stratified split sizes (seed = 42) 32

5.1 Realized feature counts, stratified split sizes (seed = 42), and class-weight ratios 42
5.2 Best XGBoost hyperparameters after 50 Optuna TPE trials on each validation set 43
5.3 Best TabM hyperparameters after 50 Optuna TPE trials on each validation set 43
5.4 Best FT-Transformer hyperparameters after 50 Optuna TPE trials on each validation set 44
5.5 Test-set predictive performance on Ferenc et al. (2019), 32 features . . 44
5.6 Test-set predictive performance on Viszkok et al. (2021), 54 features . 45
5.7 Test-set efficiency on Ferenc et al. (2019), 32 features 46
5.8 Test-set efficiency on Viszkok et al. (2021), 54 features 46
5.9 Precision, Recall, and F1-score on the Ferenc static-metric table: published Ferenc et al. (2019) scores versus this study's held-out test scores at threshold 0.5 50
5.10 Precision, Recall, and F1-score on the Viszkok static-plus-process table: published Viszkok et al. (2021) F-measure-optimal scores versus this study's held-out test scores at threshold 0.5 50

1.1 Ferenc Dataset Summary Statistics 56
1.2 Viszkok Dataset Summary Statistics 57

LIST OF FIGURES

3.1 Research Flowchart 12
3.2 Class Weight Application on Model Training Pipeline 18
3.3 The End-to-End Forward Pass of TabM 20
3.4 The End-to-End Pipeline of FT-Transformer 22
3.5 The End-to-End Pipeline of XGBoost 24

4.1 Train-only QuantileTransformer fitting in src/preprocessing.py. 31
4.2 Two-step stratified 60/20/20 split in src/splitting.py. 32
4.3 Class-weight computation from Equation 3.1 and the XGBoost ratio in src/class_weights.py. 33
4.4 Preprocessing orchestration that splits before normalization in src/run_preprocessing.py. 34
4.5 TabM ensemble-layer forward pass implementing Equation 3.2 in src/models/tabm_model.py. 36
4.6 Optuna TPE study with MedianPruner and early stopping in src/models/tabm_model.py. 38
4.7 Test-set PR-AUC, F2-Score, and MCC computation in src/evaluate.py. 39
4.8 CPU per-sample inference latency measurement in src/evaluate.py. . . 40
4.9 Mapping of stored JSON fields into comparison-table columns in src/compare.py. 41

INTISARI

Deteksi Kerentanan Dalam JavaScript Menggunakan Arsitektur TabM

Oleh

Rama Andhika Pratama
23/517333/PA/22175

Metrik perangkat lunak tingkat fungsi dapat digunakan untuk mengklasifikasikan fungsi JavaScript sebagai rentan atau tidak rentan. Dataset Ferenc et al. (2019) dan Viszkok et al. (2021) yang menyediakan metrik tersebut sebelumnya dievaluasi menggunakan model machine learning klasik dan deep learning konvensional. Penelitian ini mengimplementasikan dan mengevaluasi tiga arsitektur pada kedua dataset tersebut, yaitu Gradient Boosted Decision Tree (XGBoost) sebagai baseline machine learning, FT-Transformer sebagai baseline deep learning berbasis attention, dan TabM sebagai model deep learning berbasis ensembel yang efisien secara parameter. Seluruh model dilatih menggunakan pipeline yang sama, yaitu pembagian data stratifikasi 60/20/20, normalisasi kuantil yang dilatih hanya pada data latih, pembobotan kelas untuk menangani ketidakseimbangan kelas, dan penyetelan hyperparameter menggunakan Optuna dengan PR-AUC sebagai target optimasi. Performa diukur pada data uji menggunakan PR-AUC, F2-Score, dan Matthews Correlation Coefficient (MCC), sedangkan efisiensi diukur melalui latensi inferensi, waktu pelatihan ulang, ukuran model, dan penggunaan memori puncak. XGBoost memperoleh PR-AUC dan MCC tertinggi pada kedua dataset (PR-AUC sebesar 0,8039 pada Ferenc dan 0,9071 pada Viszkok). Pada dataset Ferenc, TabM menempati peringkat kedua setelah XGBoost pada ketiga metrik performa, di atas FT-Transformer. Pada dataset Viszkok, FT-Transformer memperoleh F2-Score tertinggi (0,8193), melampaui XGBoost (0,8091), sedangkan XGBoost tetap unggul atas FT-Transformer pada PR-AUC dan MCC. XGBoost memiliki biaya komputasi terendah pada keempat metrik efisiensi di kedua dataset, diikuti TabM, kemudian FT-Transformer. Seluruh hasil berasal dari satu random seed.

ABSTRACT

Javascript Vulnerability Detection using TabM Architectures

By

Rama Andhika Pratama
23/517333/PA/22175

Function-level software metrics can be used to classify JavaScript functions as vulnerable or non-vulnerable. The datasets of Ferenc et al. (2019) and Viszkok et al. (2021), which provide these metrics, were originally evaluated with classical machine learning and conventional deep learning models. This study implements and evaluates three architectures on both datasets: a Gradient Boosted Decision Tree (XGBoost) as the machine learning baseline, FT-Transformer as an attention-based deep learning baseline, and TabM, a parameter-efficient ensemble-based deep learning model. All models were trained under the same pipeline: a stratified 60/20/20 split, quantile normalization fitted on the training set, class weighting for class imbalance, and Optuna hyperparameter tuning with PR-AUC as the target. Performance was measured on the held-out test set using PR-AUC, F2-Score, and Matthews Correlation Coefficient (MCC), and efficiency using inference latency, retraining time, model size, and peak memory. XGBoost obtained the highest PR-AUC and MCC on both datasets (PR-AUC of 0.8039 on Ferenc and 0.9071 on Viszkok). On Ferenc, TabM ranked second behind XGBoost on all three performance metrics, ahead of FT-Transformer. On Viszkok, FT-Transformer obtained the highest F2-Score (0.8193), above XGBoost (0.8091), while XGBoost remained ahead of it on PR-AUC and MCC. XGBoost had the lowest computational cost on all four efficiency metrics on both datasets, followed by TabM and then FT-Transformer. All results come from a single random seed.

CHAPTER I

INTRODUCTION

1.1 Background

Modern systems are commonly developed using JavaScript, which makes the language a frequent target for cyber-attacks. When security flaws exist in a system, they can open the door to data breaches, which may lead to significant economic damage. As an illustration, software vulnerabilities cost the US economy an estimated $60 billion annually in productivity loss and patch redeployment, with a single cyber-attack causing damages of up to $8.75 billion (Anwar et al., 2020). The shift toward Continuous Integration and Continuous Deployment (CI/CD) can raise these risks further. To maintain a high-speed environment in CI/CD, highly sensitive credentials are often stored in a single, centralized location. If attackers compromise it, they may gain access to the entire deployment infrastructure (Pan et al., 2024). As development cycles become faster, traditional security methods struggle to keep up the pace.

Numerous efforts have been made to create automated vulnerabilities detection to keep pace with the faster development cycles. Traditional static analysis tools were originally built to address software security risk issues automatically. However, these tools often produce large volumes of raw warnings with high false-positive rates. Therefore, developers had to manually review the code again, which defeats the purpose of having an efficient automated code detector (Cheirdari and Karabatis, 2018). Machine learning and deep learning are strong candidates for vulnerability detection, since they learn vulnerability patterns directly from historical data. Classical machine learning models are lightweight and efficient, while deep learning models can learn richer representations of code, though often at higher computational cost (Harer et al., 2018).

However, raw source code usually cannot be fed directly into learning models. It must first go through a representation phase, which transforms it into a structured format that the model can process, such as graphs, tables, or tokens (Samoaa et al., 2022). Among these techniques, tabular representations are often considered the most efficient because they use readily available metrics or metadata, which suits the purpose of the CI/CD pipeline discussed previously (Prathuri et al., 2026). Fer-

enc et al. (2019) constructed a tabular JavaScript Vulnerability Dataset where they extracted the software metrics of diverse corpus from vast amounts of open-source JavaScript repositories into static source code metrics. The dataset was later expanded by Viszkok et al. (2021) where they added historical process metrics to the original dataset. Both studies applied a limited amount of learning models to evaluate their datasets. This leaves a research gap for further exploration of how various learning models process their specific tabular data of software metrics.

Recent advances in native tabular deep learning have produced architectures that perform strongly on tabular data. One such architecture, FT-Transformer (Feature Tokenizer + Transformer), proposed by Gorishniy et al. (2023) is a general-purpose baseline for standard supervised classification and regression tasks on tabular data. It adapts the Transformer architecture by transforming all features into embeddings and applying a stack of Transformer layers, allowing every layer to operate at the feature level of one object. However, despite its strong predictive performance, FT-Transformer has been noted to be computationally expensive, as the self-attention mechanism scales poorly if the number of features and training samples adds up in the long run.

To address this, Gorishniy et al. (2025) later introduced TabM, a parameter-efficient tabular deep learning architecture that uses an ensembling mechanism allowing a single neural network to simulate an ensemble of MLPs. Rather than relying on attention across features, TabM produces multiple predictions simultaneously while sharing the majority of the model's underlying parameters, and is reported to achieve competitive accuracy at a fraction of the computational cost. Given these properties, TabM is a promising candidate for JavaScript vulnerability prediction, where predictive performance and computational efficiency are both important.. Its lightweight nature may make it particularly well-suited for integration within rapid, high-throughput CI/CD pipelines, where inference speed and low overhead are as important as predictive reliability.

Given that prior studies only evaluated a limited amount of learning models on their datasets, it remains unknown how modern tabular deep learning architectures perform on software metrics data. Therefore, this research proposes the implementation and the evaluation of TabM alongside with FT-Transformer on JavaScript vulnerability detection using the tabular software metrics dataset. The two architectures are evaluated based on predictive performance and computational cost, in order to determine whether these models can offer practical application in rapid CI/CD envi-

ronments.

1.2 Problem Formulation

The current approaches for JavaScript vulnerability detection, particularly on tabular software metrics dataset, are limited to classical machine learning and deep learning models. This study addresses that gap by applying modern deep learning model, TabM, a recent native parameter-efficient tabular deep learning architecture. The model will be evaluated alongside FT-Transformer as a deep learning baseline and a Gradient Boosted Decision Tree (GBDT) Model, XGBoost, as a strong machine learning baseline to answer one main question: Does TabM offer any advantage over FT-Transformer and XGBoost for JavaScript vulnerability prediction using software metrics, in performance and cost?

1.3 Research Limitations

To maintain a focused and achievable research objective, the scope and limitations of this study are defined as follows:

1. The scope is exclusively limited to the JavaScript programming language and will not be extended to other languages such as C/C++ or Python.

2. This study is strictly limited to the tabular datasets constructed by Ferenc et al. (2019) and Viszkok et al. (2021). This constraint is necessary because of the current scarcity of metric-based JavaScript vulnerability datasets. While various vulnerability datasets exist, most of them provide raw source code instead of structured numerical features. As of June 2026, the Ferenc and Viszkok datasets are the only publicly available benchmarks, to the best of our knowledge, that fit the requirements of this research.

3. The comparison is confined to the numerical features within the tabular datasets, specifically the software metrics. Although the source code of every data exists within the datasets, it will not be utilized nor applied to the training process.

4. The evaluation is strictly bounded to three specific architectures: a Gradient Boosted Decision Tree (GBDT) machine learning (XGBoost), attention-based tabular deep learning (FT-Transformer), and parameter-efficient ensemble-based

deep learning (TabM). The study will not explore or implement other architectures.

1.4 Research Objectives

The specific objectives of this research are:

1. To implement and evaluate the three different architectures—a Gradient Boosted Decision Tree, an attention-based deep learning architecture (FT-Transformer), and a parameter-efficient ensemble architecture (TabM)—on the JavaScript tabular software metrics datasets provided by Ferenc et al. (2019) and Viszkok et al. (2021).
2. To benchmark the predictive performance of the three architectures to determine which model most accurately and effectively captures vulnerability patterns within the datasets.
3. To determine whether TabM can deliver a favorable balance between predictive accuracy and computational efficiency, making it a practical option for real-world vulnerability detection pipelines.

1.5 Research Benefits

The benefits derived from this research are divided into two categories:

1. **Theoretical Benefits:** This study contributes empirical evidence to the under-explored intersection of native tabular deep learning and software vulnerability detection. By benchmarking the different architectures, this research provides a reproducible academic framework. It clarifies the theoretical viability and architectural trade-offs of using deep learning to capture complex vulnerability patterns using software metrics.
2. **Practical Benefits:** The findings of this research offer guidance to software engineers, security tool developers, and DevOps teams seeking automated vulnerability detection methods for their pipeline environment. By evaluating the trade-offs between the different architectures, they can determine whether the computational cost of deploying richer, complex representations is justified, or

if a lightweight, efficient model provides a more practical balance of speed and accuracy for their high-speed environments.

CHAPTER II

LITERATURE REVIEW

2.1 Tabular Software Metrics and Vulnerability Detection

When implementing vulnerability detection models for software engineering, tabular data remains highly relevant since they are lightweight. This format often utilizes metric-based features that were extracted from the source code and their metadata from various repositories. Although the efficiency depends on the feature extraction method itself, it bypasses the need for extensive feature engineering, making it lightweight than other feature representations methods. The JavaScript vulnerability datasets introduced by Ferenc et al. (2019) and Viszkok et al. (2021) rely on metric-based features and serve as the benchmark that demonstrates the efficacy of the lightweight tabular dataset in this study.

In the world of tabular data processing, Gradient Boosted Decision Tree (GBDT) algorithms such as XGBoost remain as robust baselines. Their strong performance came from their intrinsic ability to handle diverse and heterogeneous feature distributions through successive space partitioning when handling data (Ye et al., 2025). While GBDTs remain highly effective, modern deep learning architectures are increasingly narrowing this performance gap by demonstrating a improvement to map higher-order interactions and complex non-linear feature compositions within tabular datasets (Ye et al., 2025).

However, the narrowing performance gap between deep learning and GBDTs does not mean that every deep learning architecture will perform well on every dataset and task. It is still necessary to identify an architecture that is appropriate for the specific data type and problem at hand. Architectures that are originally designed for specific other data types, such as spatial locality in images or sequential order in text, carry assumptions that do not naturally hold for tabular data, where the features are heterogeneous and have no inherent ordering or spatial relationship.

This leaves a gap for further exploration of deep learning approaches that are genuinely designed for tabular data, while also being efficient enough to support the fast-paced nature of CI/CD pipelines. TabM is reported as a method that offers both of these requirements. Instead of introducing heavy computational overhead, TabM uses parameter-efficient ensembling, where a single model behaves like an ensem-

ble of independent networks that share most of their parameters, producing multiple predictions at once without significant memory costs (Gorishniy et al., 2025). This makes TabM a well-suited choice for the lightweight and real-time demands of the vulnerability detection task in this study.

2.2 Comparative Analysis of Previous Vulnerability Detection Models

The foundation for lightweight JavaScript vulnerability prediction using software metrics was established by Ferenc et al. (2019) and subsequently expanded by Viszkok et al. (2021). By combining static source code features with historical process metrics into a standardized software metrics tabular dataset, they provided a highly lightweight and efficient alternative to complex code analysis, significantly advancing the field of automated vulnerability detection. However, to the best of our literature search, their methodologies are limited and relied exclusively on classical machine learning models (such as Random Forest) and deep learning algorithms (such as Deep Neural Network). This leaves a gap for future exploration where modern learning models could further improve predictive performance on such tabular software metrics datasets.

Several studies have attempted to build upon these datasets. Sheneamer (2024) expanded the exploration by applying a Convolutional Neural Network (CNN). However, this approach is fundamentally mismatched, as CNNs are spatially biased models better suited for image data and tabular features lack spatial relationships (Ingrosso and Goldt, 2022). Kalouptsoglou et al. (2022) extended the exploration further by combining the software metrics with text features mined directly from source code into a single unified model. However, their results did not achieve higher results. Mosolygo et al. (2021) took a different direction by constructing Word2Vec vector similarity with custom filtering rules, offering built-in explainability and the ability to provide evidence for each flagged instance. However, their approach did not incorporate the software metrics component of the dataset and does not align with our research.

Other studies have explored advanced deep learning approaches. However, most of them suffer from being computationally expensive, which makes them unsuitable for high-speed CI/CD environments. Shwartz-Ziv and Armon (2022) demonstrated that combining deep learning models with XGBoost outperformed individual deep models, standalone XGBoost, and classical ensembles alike, but it requires a

large amount of computational resources. Similarly, Gorishniy et al. (2023) proposed the Feature Tokenizer Transformer (FT-Transformer), which produces richer and deeper feature representation. However, it still remains computationally expensive for lightweight deployment. The proposed approach in this study therefore aims to identify models that balance predictive accuracy with computational efficiency, making them practical for JavaScript vulnerability detection within rapid CI/CD pipelines. A comparative analysis of the algorithms, datasets, methods, and their respective advantages and disadvantages is summarized in Table 2.1.

Table 2.1: Comparison of Recent Vulnerability Prediction Methodologies

Author & Year	**Algorithm / Architecture**	**Dataset Size & Type**	**Imbalance Handling**	**Advantages and Disadvantages**
Ferenc et al. (2019)	8 ML algorithms, including KNN, Random Forest and 2 DL algorithms	12,125 JavaScript functions using only static source code metrics	Tested random under-sampling and over-sampling at various ratios	(+) Introduced the foundational tabular dataset mapping JS source code to static software metrics. (-) Relied on traditional classical machine learning and deep learning models
Viszkok et al. (2021)	8 ML algorithms, including KNN, Random Forest and 2 DL algorithms	8,038 JavaScript functions using static and process metrics	Tested random under-sampling and over-sampling at various ratios	(+) Expanded the tabular dataset by adding historical process metrics. (-) Still relied on traditional classical machine learning and deep learning models
Sheneamer (2024)	Stacked 1D CNNs including VGG16, VGG19, AlexNet, and ResNet	30,367 JavaScript functions across 3 tabular metric datasets (Ferenc et al., Viszkok et al., and Apache Tomcat)	Applied random under-sampling and random over-sampling techniques	(+) Expanded the exploration of the dataset by applying an advanced neural network method. (-) Relies on spatially biased CNNs

Table 2.1 (continued)

Author & Year	**Algorithm / Architecture**	**Dataset Size & Type**	**Imbalance Handling**	**Advantages and Disadvantages**
Shwartz-Ziv and Armon (2022)	Compared 4 tabular deep learning models (TabNet, DNF-Net, NODE, 1D-CNN) against XGBoost	11 diverse tabular datasets containing up to 1,000,000 samples	Not explicitly addressed	(+) The ensemble combining both deep learning models and XGBoost outperformed individual deep models, standalone XGBoost, and ensembles made strictly of classical models. (-) Demonstrated the ensemble of XGBoost and deep models are computationally expensive
Gorishniy et al. (2023)	Proposed FT-Transformer and ResNet architectures for tabular data	11 diverse public tabular datasets up to 1,200,192 samples	Not explicitly addressed in the methodology	(+) Richer and deeper representation when fed more features (-) FT-Transformer demands high computational resources
Kalouptsoglou et al. (2022)	ML, DL (CNN), and ensemble models	Processed Ferenc et al. (2019) Dataset: 12,106 open-source JavaScript functions	Random over-sampling and under-sampling	(+) Extended the exploration further by combining the original software metrics and the text features mined from the source codes into a unified model. (-) The combined models did not achieved higher result than the original study

Table 2.1 (continued)

Author & Year	**Algorithm / Architecture**	**Dataset Size & Type**	**Imbalance Handling**	**Advantages and Disadvantages**
Mosolygo et al. (2021)	Word2vec vector similarity with custom filtering rules	Ferenc et al. (2019) Dataset processed into 186 real-world vulnerability fixes from 91 JavaScript projects	Inherently unaffected; relies strictly on similarity to reference lines	(+) Has built-in explainability and able to provide proof for each flagged data. (-) The model did not rely on the tabular software metrics data

2.3 Identified Research Gap and Significance of the Study

As synthesized in Table 2.1, a critical gap exists in the current application of advanced machine learning and deep learning methods for JavaScript vulnerability detection using tabular software metrics datasets. Several studies have attempted to address this by building more advanced models on the dataset. However, these efforts still remain limited. Several approaches are fundamentally mismatched for tabular data due to their reliance on spatially biased architectures, others are misaligned with the research scope as they disregard the software metrics feature entirely, and some simply fail to achieve an improvement over existing baselines. On the other hand, studies that have applied advanced deep learning to tabular datasets more broadly still share a common limitation: expensive computational cost, which makes them impractical for deployment in high-speed CI/CD pipelines where lightweight and efficient is essential.

To address this gap, this study proposes TabM, a recent native tabular method designed for processing tabular data efficiently, and evaluates it on both tabular datasets. Introduced by Gorishniy et al. (2025), TabM represents a significant advancement in native tabular deep learning through its use of parameter-efficient ensembling, which allows a single model to function as multiple neural networks simultaneously without demanding excessive memory or computation. This property may make TabM well-suited for integration into rapid CI/CD pipelines. To benchmark its performance, TabM is compared against two baselines. The FT-Transformer serves as the deep learning baseline, offering richer and deeper feature representations, though it remains computationally expensive. A Gradient Boosted Decision Tree (GBDT) model is also included as the machine learning baseline, representing the established strong

baseline for tabular data processing, in order to assess whether modern deep learning approach can compete with, or surpass, the dominant approach for structured tabular data.

CHAPTER III

RESEARCH METHODOLOGY

3.1 General Description

This chapter describes the complete structure of the research methodology employed in this study. The research follows a structured pipeline designed to produce a fair and reproducible comparative evaluation. At a high level, the workflow proceeds through nine sequential stages: (1) data acquisition, (2) data preprocessing, (3) stratified data split, (4) class imbalance handling on training set, (5) model implementation, (6) hyperparameter tuning, (7) model training, (8) model evaluation, and (9) comparative analysis. Figure 3.1 presents the research flowchart that illustrates the research's end-to-end pipeline. The process begins with data acquisition, explaining about the characteristics of each dataset. The datasets are then preprocessed and partitioned into training, validation, and test sets. Before the models are configured, the imbalance inside the datasets will be adjusted using class weight. The models are then configured, tuned, and trained using the specified evaluation protocol. Finally, the models are evaluated on the held-out test set using a common suite of classification metrics, and their results are compared.

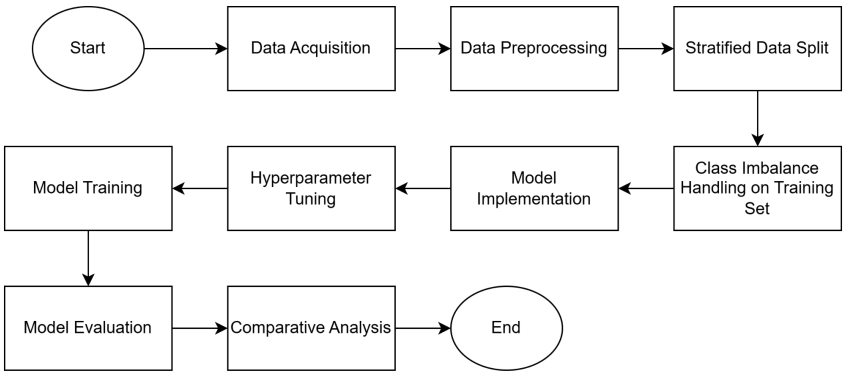


Figure 3.1: Research Flowchart

3.2 Data Acquisition

3.2.1 Ferenc et al. (2019) Dataset

The first dataset was constructed by Ferenc et al. (2019) as part of a study investigating whether static source code metrics could serve as effective predictors of security vulnerabilities in JavaScript functions. To build the dataset, the authors mined two publicly available vulnerability databases, the Node Security Platform (nsp) and the Snyk Vulnerability Database. Then, they cross-referenced their entries with GitHub repositories to locate the specific commits that fixed each reported vulnerability. For each vulnerability, the corresponding patch file was extracted and mapped to the JavaScript functions it modified. Functions in the pre-fix version of the code (i.e., before the security patch was applied) that fell within the range of the patch's changed lines were marked as vulnerable, while all remaining functions in those same projects were marked as non-vulnerable. Test functions, identified by their location under "test" folders, were excluded to prevent them from distorting the prediction models. This process resulted in a dataset of 12,125 JavaScript functions drawn from over 300 open-source Node.js projects, of which 1,496 (approximately 12.3%) are labeled as vulnerable and 10,629 as non-vulnerable and 35 available static source code metrics.

For each function, 35 static source code metrics were computed using two complementary static analysis tools: OpenStaticAnalyzer and escomplex. Static source code metrics are structural properties derived by analyzing the source code at a single point in time without executing it. The metrics captured span several dimensions of code quality and complexity, including size metrics such as Lines of Code (LOC) and Logical Lines of Code (LLOC), complexity metrics such as Cyclomatic Complexity (McCC/CYCL) which counts the number of independent execution paths through a function, nesting metrics such as Nesting Level (NL), structural metrics such as the Number of Parameters (NUMPAR/PARAMS), clone-related metrics such as Clone Coverage (CC) and Clone Instances (CI), comment and documentation metrics such as Comment Density (CD) and Comment Lines of Code (CLOC), and the full suite of Halstead software science metrics — including Halstead Volume (HVOL), Halstead Difficulty (HDIFF), Halstead Effort (HEFF), and Halstead Bugs (HBUGS) — which characterize the information content and estimated error proneness of the code based on its operator and operand counts.

The target label is a binary indicator stored in the final column of the dataset.

A value of 1 denotes that the function was vulnerable, meaning it was directly modified by a known security-fixing commit prior to the fix being applied. A value of 0 denotes that the function was non-vulnerable, meaning it was not implicated in any vulnerability fix within the same codebase. This binary labeling scheme makes the dataset directly suitable for supervised binary classification.

3.2.2 Viszkok et al. (2021) Dataset

To determine the performance of the models on different sets, this study also employs a second dataset provided by Viszkok et al. (2021). This dataset was originally constructed by Ferenc et al. (2019) and was expanded by Viszkok et al. to include historical process data derived from each project's version control history.

The dataset contains function-level records extracted from the same corpus of over 300 open-source JavaScript repositories used in the original study. However, because the extraction of process metrics requires a complete and accessible Git history for each project, not all functions from the original 12,125-function dataset could be retained — projects whose repositories were no longer accessible or whose version control history was incomplete had to be excluded. As a result, the extended dataset contains 8,038 functions, a reduction of roughly 33% from the original. This creates a trade-off that adding historical data gives the model more depth and features to work with, but it shrinks the overall size of the dataset.

For each function, the extended dataset preserves the original 35 static source code metrics. This count added seven additional metrics in the Ferenc et al. dataset because Viszkok et al. incorporated additional static metrics during the extension process, bringing the static metric total from 35 to 42. In addition, the dataset introduces 19 process metrics, also referred to as historical change or process metrics. It captures the development history of each function as recorded in its Git commit log. These include the Number of Modifications (NOMOD), which counts how many times a function was changed over its lifetime; the Number of Contributors (NOContr), which captures how many distinct developers modified it; Code Churn (CChurn), defined as the cumulative sum of lines added minus lines deleted; the Average Time Between Changes (AVGTBC); and various aggregated counts of added, deleted, and modified lines across commits. Together, the 42 static and 19 process metrics create an available combined feature space of 61 predictors per function, making this a substantially richer representation than the original dataset.

This extended dataset serves as the second dataset for the experiments in this study. The richer feature set is expected to improve model performance, which was confirmed in the original study where Viszkok et al., reported significant gains in F-measure, precision, and recall when process metrics were added. Although the reduction in training samples may introduce its own flaws. The binary target label remains identical to that of the Ferenc et al. dataset: a value of 1 indicates that the function was vulnerable prior to a known security fix, while a value of 0 indicates that it was non-vulnerable. The class distribution also remains heavily imbalanced, with vulnerable functions constituting a minority of the total records.

3.3 Data Preprocessing

To ensure a fair and rigorous comparison, a standardized preprocessing pipeline is applied uniformly across all three architectures. Because the features derived from both datasets are strictly continuous numerical metrics, categorical encoding is not required. The preprocessing pipeline consists of the following three sequential steps:

- **Feature Selection / Filtering:** Identifier columns (*name, longname, path, full_repo_path, hash*, etc), zero-variance features (*CLLC*, etc), and one feature from each near-identical collinear pair (*HBUGS with HVOL, HTIME with HEFF*, etc) are removed from the dataset. These features carry no predictive signal and their removal are necessary for standard data hygiene practice rather than a model-specific decision, ensuring that all three architectures benefit equally from a cleaner feature space.

- **Positional Columns Isolation:** Positional features such as *line, column, endline,* and *endcolumn* are excluded from model training. Although these columns are moderately correlated to the target variable (*Vuln* ≈ 0.20), this relationship is likely an artifact of the dataset's construction rather than a generalizable signal. Retaining them may lead to all models exploiting the dataset construction artifact rather than actual vulnerability patterns, inflating performance scores and weakening the validity of the comparison. However, unlike the previous preprocessing step, the features are kept for further observation. The target column is also isolated to control exposures between the data splits and which pipeline can access the target label.

- **Symmetrical Feature Normalization:** Neural networks are sensitive to feature scale, skew, and outliers. In order to maintain well-conditioned input representation, QuantileTransformer is applied to all numerical features for all three models and fit on training data only. Because tree-based models are invariant to monotonic feature transformations, this only affects TabM and FT-Transformer, giving them a standard, well-conditioned input representation. We note this does not guarantee equal performance potential across model families.

3.4 Data Splitting

The dataset is partitioned into three non-overlapping subsets: a training set, a validation set, and a test set. The split is performed using stratified sampling to preserve the original class ratio in all three subsets. Table 3.1 presents the data split configuration for Ferenc's dataset and Table 3.2 presents the data split configuration for Viszkok's Dataset.

Table 3.1: Data Split Configuration for Ferenc's Dataset

Dataset Split	**Total Samples**	**Vulnerable**	**Non-Vulnerable**
Training (60%)	∼7,275	∼898	∼6,377
Validation (20%)	∼2,425	∼299	∼2,126
Test (20%)	∼2,425	∼299	∼2,126
Total	12,125	1,496 (12.34%)	10,629 (87.66%)

Table 3.2: Data Split Configuration for Viszkok's Dataset

Dataset Split	**Total Samples**	**Vulnerable**	**Non-Vulnerable**
Training (60%)	∼4,823	∼576	∼4,247
Validation (20%)	∼1,608	∼192	∼1,416
Test (20%)	∼1,608	∼192	∼1,416
Total	8,038	960 (11.94%)	7,078 (88.06%)

The 60/20/20 stratified split is selected because of the class imbalance in both datasets. Since the minority class makes up only about 12% of the data, an 80/10/10 split would yield only 96 vulnerable samples in each evaluation set of Viszkok's dataset, producing a 95% confidence interval width of ± 0.103, which is too wide to support reliable comparisons between models. A 60/20/20 split instead provides

192 and 299 vulnerable samples in the validation and test sets of the Viszkok and Ferenc datasets respectively, narrowing the interval to ± 0.071 and ± 0.057. This choice trades training volume for evaluation reliability, leaving approximately 7,275 and 4,823 training samples for the Ferenc and Viszkok datasets. This trade-off applies equally to all three architectures, and the effect of training set size on each model is noted as a limitation of this study. All splits use a fixed random seed (seed = 42) to ensure reproducibility.

3.5 Class Imbalance Analysis and Handling

The dataset suffers from a severe class imbalance, with approximately 12% of samples representing vulnerable functions and the remaining 88% representing non-vulnerable functions. This inequality poses a significant challenge when doing model training. A naive classifier could predict every sample as non-vulnerable and it would still achieve an 88% accuracy while completely failing at the intended detection task.

To address this issue, this study employs class weight adjustment, a technique grounded in cost-sensitive learning (Elkan), to mitigate the class imbalance problem. Cost-sensitive learning modifies the training objective so that the algorithm penalizes the model for misclassifying a minority sample more than a majority sample. Consequently, the loss function for each model is modified to assign a proportionally higher penalty to the vulnerable class. The weight assigned to the minority class is calculated using the equation below:

$$W_{\text{minority}} = \frac{N_{\text{total}}}{C \times N_{\text{minority}}} \tag{3.1}$$

where N_{total} is the total number of samples, N_{minority} is the number of vulnerable samples, and C is the number of classes, which in this case would be equal to 2 because our task is binary classification. This method was chosen over SMOTE and oversampling methods because they distort feature space differently for different model architecture making an unfair comparison environment. In addition, for imbalanced tabular software metrics, retaining the original class imbalance and training standard classifiers without synthetic oversampling can be competitive or superior to SMOTE-based rebalancing, which may fail to adequately represent the minority distribution (Sakho et al., 2026).

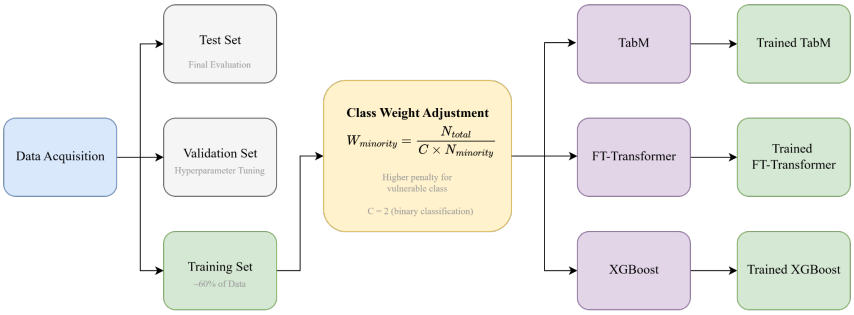


Figure 3.2: Class Weight Application on Model Training Pipeline

3.6 Model Implementation

3.6.1 TabM (Native Tabular Deep Learning)

Each JavaScript function in the Ferenc et al. (2019) and Viszkok et al. (2021) datasets enters the TabM model as a single standardized numerical vector $x \in \mathbb{R}^n$, where n is the number of software metrics available in each respective dataset. Each of the n numerical metrics is first passed through a non-linear numerical feature embedding, following the way Gorishniy et al. (2025) equip TabM with embeddings: a modified version of the piecewise-linear embeddings with quantile-based bins. The embedded metrics are flattened into a single vector $x_{\text{emb}} \in \mathbb{R}^{(n \cdot d_{\text{e}})}$, where d_{e} is the embedding size, and this vector is what the backbone receives. This study uses the embeddings because the paper reports that TabM mini with these embeddings outperforms TabM without them, both in average improvement over MLP (2.92% vs. 2.15%) and in mean rank (1.7 vs. 2.8), and because the aim of this study is to reimplement the paper's strongest model. At the start of the forward pass, the embedded vector is arranged into a parallel batch $X \in \mathbb{R}^{k \times (n \cdot d_{\text{e}})}$ with one row per implicit ensemble member. At inference, each row is an identical copy of the embedded input. During training, the rows of X may hold k different training objects (the default in TabM) or the same objects (the shared-batch variant). This arrangement is the foundation of TabM's parameter-efficient ensembling: it allows all k implicit ensemble members to be processed simultaneously in one forward pass, eliminating the need for k separate model executions.

Rather than building k entirely separate networks, all k input streams are routed through a single MLP backbone whose primary weight matrix $W \in \mathbb{R}^{d \times d}$ is shared across all members and handles the main computational work (following

the paper, layer widths are simplified to $d_1 = d_2 = d$). To introduce diversity between members without duplicating W, each ensemble member $i \in \{1, \ldots, k\}$ is assigned a set of lightweight, non-shared adapter vectors r_i, s_i, $b_i \in \mathbb{R}^d$. The linear transformation for member i at any given layer is computed as:

$$\ell_i(x_i) = r_i \odot \big(W(s_i \odot x_i)\big) + b_i \tag{3.2}$$

where $\odot$ denotes the Hadamard (elementwise) product, r_i is the input adapter that scales the incoming representation before it is multiplied by W, s_i is the output adapter that scales the result, and b_i is a member-specific bias vector. In the first linear layer, the input adapter r_i has the dimension of the flattened embedded input ($n \cdot d_\mathrm{e}$) instead of d, and is initialized from the standard normal distribution, as in the paper's embedding variants. This formulation is mathematically equivalent to giving each member its own unique weight matrix $W_i = W \odot (s_i r_i^\mathsf{T})$, but without explicitly constructing it. Each additional member adds only $3d$ parameters per layer, so the per-member overhead is $O(d)$ rather than the $O(d^2)$ of a separate weight matrix. Each of the k streams therefore receives a distinctly transformed view of the n input metrics, achieving representational diversity at negligible additional memory cost.

After the linear transformation at each layer, standard non-linear operations are applied to each of the k streams strictly independently. Specifically, a ReLU activation is applied elementwise to each stream's output, followed by Dropout regularization applied separately to each stream's activated representation. This per-stream independence is essential: because the non-linearities and stochastic masking are never shared across members, each of the k simulated ensemble members traces a distinct, non-linear pathway through the n-dimensional software metric space across all N hidden blocks of the backbone. This separation ensures that the ensemble members learn complementary feature combinations from the static and process metrics, which the final aggregation step then leverages to produce more robust predictions than any single member could achieve on its own.

After passing through all N hidden blocks, each of the k parallel streams is fed into its own non-shared linear prediction head, producing k separate scalar outputs $\{\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_k\}$. Each output is passed through a Sigmoid activation function, $\sigma(\hat{y}_i) = (1+e^{-\hat{y}_i})^{-1}$, which maps it to the interval $(0, 1)$ and gives member i's vulnerability probability p_i. During training, each member's prediction is optimized with its own loss (binary cross-entropy against the label y), and the objective is the mean of

these k individual losses, $\frac{1}{k}\sum_{i=1}^{k} L(p_i, y)$. At inference, the k member predictions are aggregated by computing the arithmetic mean of their probabilities, yielding a single unified probability $\bar{p} = \frac{1}{k}\sum_{i=1}^{k} p_i$ for the input function. A classification threshold of 0.5 is then applied: if $\bar{p} \geq 0.5$, the function is classified as vulnerable (label = 1); otherwise, it is classified as non-vulnerable (label = 0). A single sigmoid output with a fixed 0.5 threshold is used so that all models in this study are evaluated under the same decision rule, which keeps the reported classification metrics reflecting differences between the models rather than differences in how their outputs were combined. The complete end-to-end data flow described across all four stages, from input expansion through shared backbone processing to final aggregation, is illustrated in Figure 3.3.

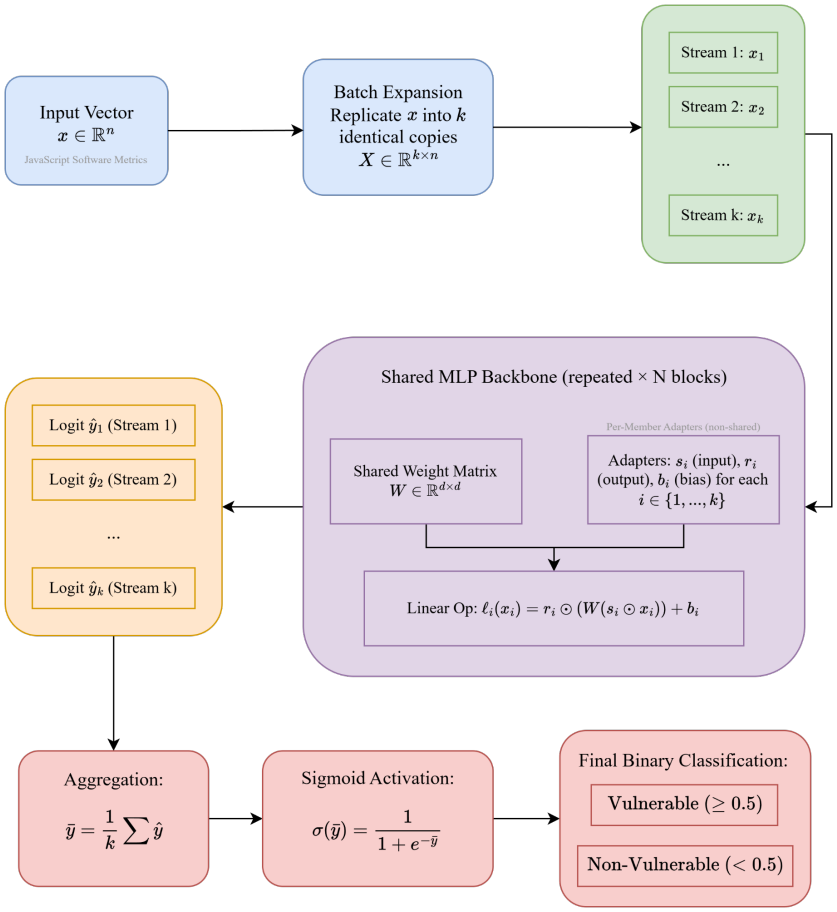


Figure 3.3: The End-to-End Forward Pass of TabM

3.6.2 FT-Transformer

FT-Transformer is included in this study as the attention-based deep learning baseline for tabular vulnerability prediction. Its defining design choice is that every software metric is embedded as its own token and processed with self-attention, so the model can learn how metrics interact with one another instead of receiving them as one flat input vector. This sets it apart from the other models in this study, such as XGBoost as a tree-based model and TabM as a parameter-efficient deep learning alternative. The cost of this design scales with the number of input features: because self-attention is computed across all feature tokens, computation and memory grow with the number of features (quadratically in the number of tokens). A larger feature set gives the model more interactions to learn, which may improve predictive performance, but it also raises training time and memory requirements. This trade-off matters for a rapid CI/CD pipeline, where detection must remain practical to run. FT-Transformer is therefore used to examine whether the performance gained from modeling feature interactions justifies its added cost, particularly when moving from the 35-feature dataset to the 61-feature dataset.

FT-Transformer, proposed by Gorishniy et al. (2023) in *Revisiting Deep Learning Models for Tabular Data*, adapts the Transformer architecture for tabular data through two sequential components: a Feature Tokenizer and a standard Transformer backbone. In this study, each JavaScript function is represented as a numerical feature vector, either 35 features from the Ferenc et al. (2019) dataset or 61 features from the Viszkok et al. (2021) dataset, depending on which dataset is being processed. The Feature Tokenizer converts each scalar feature into a dense embedding vector by applying a per-feature linear projection combined with a learnable feature bias. This transforms the raw software metric vector into a sequence of token embeddings of shape $k \times d$, where k is the number of features and d is the embedding dimension.

Once the feature embeddings are constructed, a special [CLS] (classification) token is prepended to the sequence, and the combined sequence of $k + 1$ tokens is passed through L stacked Transformer layers. Each layer applies multi-head self-attention followed by a position-wise feed-forward network, with layer normalization and residual connections following the PreNorm variant. Because every feature token attends to every other feature token at each layer, the model captures interactions across all 35 or 61 software metrics simultaneously, and this is also the source of the cost scaling with k described above. After all Transformer layers have been applied,

the final representation of the [CLS] token serves as the aggregated function-level representation. This representation is passed through a linear prediction head activated by a sigmoid function, producing the vulnerability probability for the input function. A classification threshold of 0.5 is then applied: if $\sigma(\hat{y}) \geq 0.5$, the function is classified as vulnerable; otherwise it is classified as non-vulnerable.

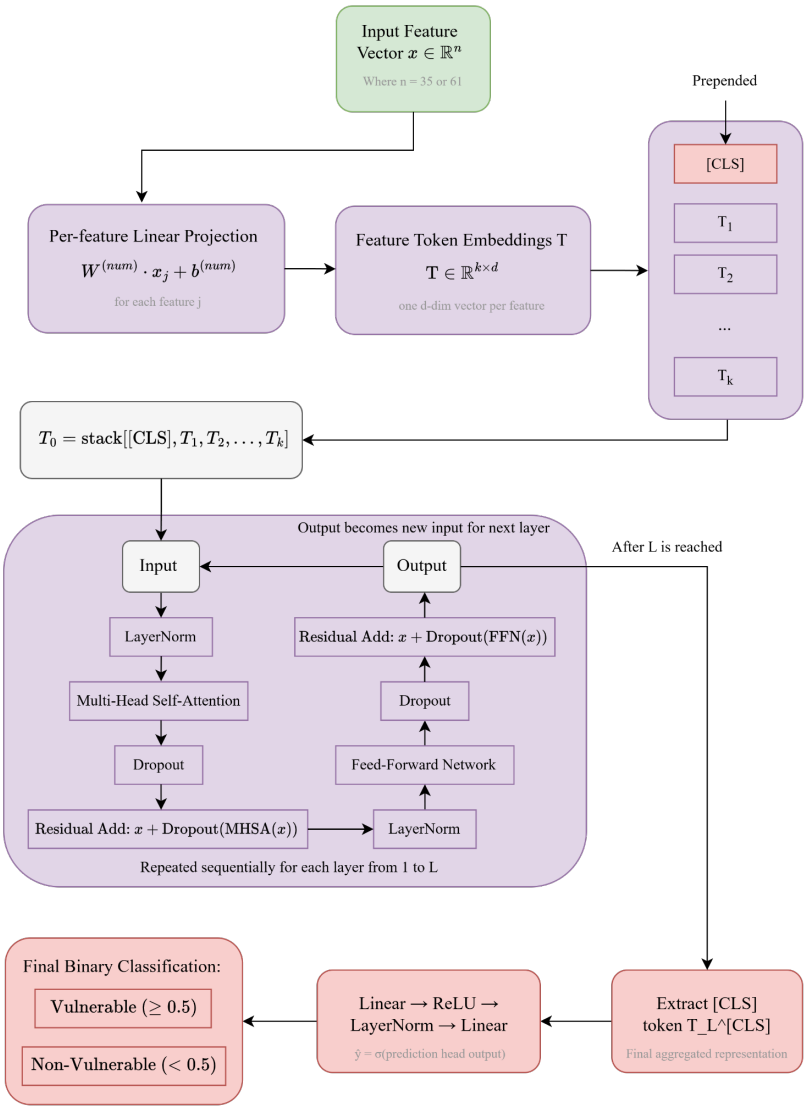


Figure 3.4: The End-to-End Pipeline of FT-Transformer

3.6.3 Gradient Boosted Decision Trees (GDBT)

XGBoost is included as the Gradient Boosted Decision Tree (GBDT) baseline for this study. GBDT models are widely regarded as strong performers on tabu-

lar data because they handle heterogeneous feature distributions through successive greedy partitioning of the feature space, and they can capture non-linear relationships between software metrics and vulnerability labels without manual feature engineering. XGBoost is therefore included as a strong baseline to examine whether any of the other models can compete with it in terms of the balance between performance and efficiency.

In this study, XGBoost receives the preprocessed feature vector of a JavaScript function as input: 35 static source code metrics for the Ferenc et al. (2019) dataset, or 61 static and process metrics for the Viszkok et al. (2021) dataset. Both are passed through a QuantileTransformer to normalize their distributions. XGBoost builds an additive ensemble of regression trees (Chen and Guestrin, 2016) within the gradient boosting framework of Friedman (2001). At each boosting iteration, a new tree is fit using the gradient and Hessian of the binary cross-entropy loss with respect to the current ensemble's log-odds predictions. For this loss, the gradient reduces to the difference between the predicted probability and the true label, so each new tree concentrates on the functions the model currently misfits. Each tree greedily selects the split variable and threshold that maximize the gain in a regularized objective. The resulting leaf weights are added to the ensemble scaled by a learning rate ν, a shrinkage strategy that improves generalization (Friedman, 2001). After M iterations, the ensemble's log-odds score is passed through the sigmoid function and thresholded at 0.5 for binary classification.

To address the severe class imbalance in both datasets, `scale_pos_weight` is set to the ratio of non-vulnerable to vulnerable training samples (approximately 7.10 for the Ferenc dataset and 7.37 for the Viszkok dataset). This upweights the gradient and Hessian of each minority-class sample, in line with the cost-sensitive objective described in Section 3.5. Hyperparameters are tuned with Optuna's TPE sampler on the fixed validation set, using PR-AUC as the optimization target with early stopping. Since the two datasets differ in feature dimensionality, each is tuned independently under the same protocol. The tuned model for each dataset is then retrained on the corresponding training set and evaluated on the held-out test set.

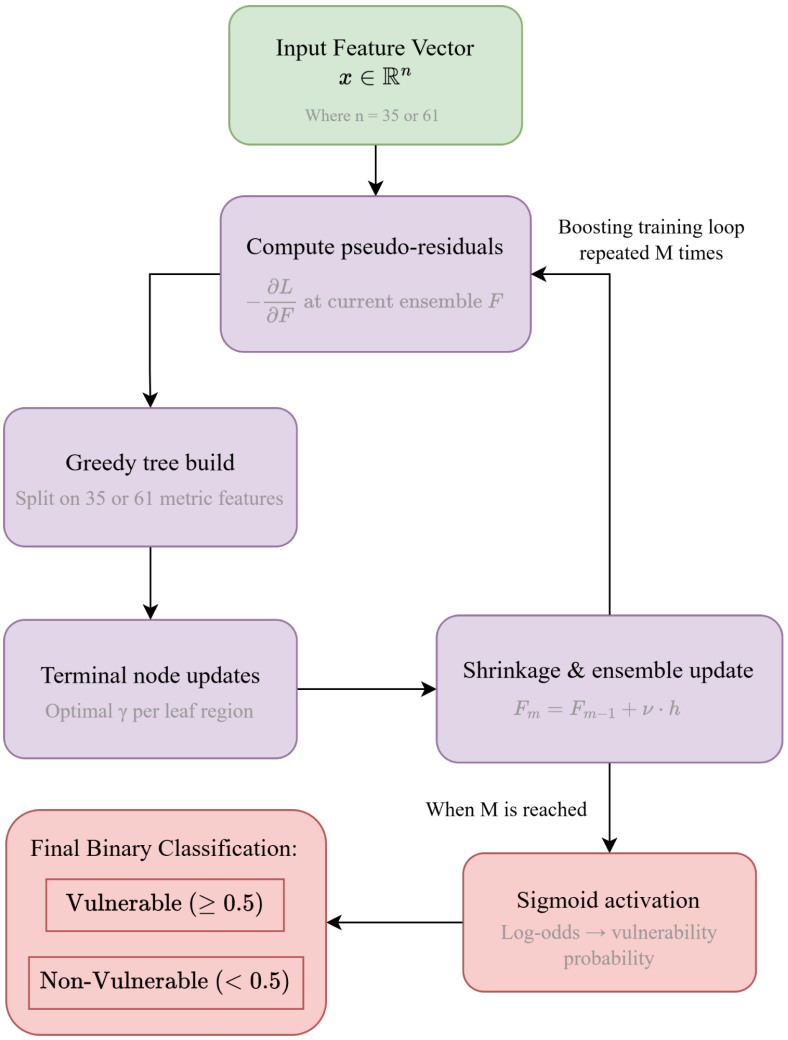


Figure 3.5: The End-to-End Pipeline of XGBoost

3.7 Hyperparameter Tuning

To ensure a fair and unbiased comparison, all three models are tuned with the same protocol: the same search algorithm, number of trials, validation set, and tuning metric. Tuning is performed exclusively on the training and validation sets, and the held-out test set is never accessed during this phase. Table 3.3 summarizes the tuning configuration for each model.

Table 3.3: Hyperparameter Tuning Configuration for All Three Architectures

Hyperparameter	**GBDT (XGBoost)**	**FT-Transformer**	**TabM**
Search Method	Optuna (TPE)	Optuna (TPE)	Optuna (TPE)
Num of Trials	50 trials	50 trials	50 trials
Validation Strategy	Fixed val set	Fixed val set	Fixed val set
Tuning Metric	PR-AUC	PR-AUC	PR-AUC
Early Stopping	50 Rounds	20 Epochs	20 Epochs
Key Parameters Tuned	n_estimators [50–2000] · max_depth [3–12] · learning_rate [0.001–0.3] · subsample [0.5–1.0] · colsample_bytree [0.5–1.0] · min_child_ weight [1–10] · reg_alpha [0–5] · reg_lambda [0–5] · scale_pos_weight fixed at 7.37 / 7.10	num_layers [1–4] · d_ token [64–512] · n_heads {4, 8} · attention_dropout [0.0–0.4] · ffn_dropout [0.0–0.4] · ffn_factor [1.0–4.0] · learning_rate [1e-5–1e-3] · weight_ decay [1e-6–1e-3] · batch_size {256, 512, 1024}	num_layers [1–4] · hidden_dim [64–512] · dropout [0.0–0.5] · learning_rate [1e-5–1e-2] · weight_decay [1e-6–1e-3] · batch_size {256, 512, 1024} · k multiplier [4–64]

Optuna with the Tree-structured Parzen Estimator (TPE) sampler is used for all three models. TPE is a probabilistic method that concentrates sampling on promising regions of the search space, which makes it more sample-efficient than grid or random search. Using the same sampler for every model ensures that no model benefits from a more capable search algorithm.

Each model is tuned with 50 trials. A fixed trial count was chosen over a fixed time budget so that every model receives the same number of search iterations regardless of its per-trial cost. The search spaces differ between models because the architectures have different hyperparameters (eight tuned parameters for XGBoost, nine for FT-Transformer, and seven for TabM; see Table 3.3), but the search effort is identical. Since tuning is performed once offline before deployment, its duration is not treated as a deployment cost and is reported only as supplementary information.

A fixed validation set is used for hyperparameter tuning and early stopping in all three models. This keeps the tuning protocol uniform: applying cross-validation to only some of the models would give them a more thorough validation than the others, and applying it to all of them would add complexity to early stopping, which requires a fixed set of held-out samples to monitor. The fixed validation set is the

same stratified partition defined in Section 3.4, so all models are tuned and stopped against identical data. A limitation of this choice is that hyperparameter selection depends on a single validation set, which contains relatively few vulnerable samples (approximately 299 and 192 for the Ferenc and Viszkok datasets, respectively).

As for the tuning metric, PR-AUC is selected for all models. Given the class imbalance of approximately 88:12 between non-vulnerable and vulnerable classes, ROC-AUC would be unsuitable, because it is heavily influenced by the number of true negatives. On the other hand, PR-AUC focuses on the precision and recall of the minority class, which is the class that actually matters in vulnerability detection. Applying the same metric across all three models ensures that all are being optimized toward the same objective.

Early stopping is applied to all models to prevent overfitting, monitoring validation PR-AUC in every case. The patience values differ because the unit of training progress differs between model families, so equal numbers would not represent equal patience. For XGBoost, each boosting round adds one tree and the validation score typically improves gradually, particularly at the low learning rates included in the search space (down to 0.001) and with up to 2,000 estimators. A patience of 50 rounds is therefore used. For TabM and FT-Transformer, each epoch is a full pass over the training data, and a patience of 20 epochs is used for both. These values are fixed in advance and are not tuned.

3.8 Model Evaluation

Given the class imbalance in our dataset, specific evaluation metrics are needed to correctly measure the performance of our models. The focus should lean towards minimizing false negatives, as a false negative that passes into the deployment pipeline is considerably more dangerous than a false positive. Ensuring that the trade-off between model performance and computational resource consumption is reasonable is also an important consideration, as it allows us to identify which model delivers strong performance while remaining lightweight enough for application in a rapid CI/CD environment. To address both concerns, the evaluation metrics are split into two distinct categories: performance metrics and efficiency metrics.

3.8.1 Performance Mentrics

PR-AUC measures the area under the Precision-Recall curve across all classification thresholds for the minority class. It summarizes the full precision-recall tradeoff on the vulnerable class without being inflated by the majority class. On the other hand, ROC AUC is susceptible to inflation caused by high true negative counts, which can make a poorly performing model appear deceptively strong. Therefore, PR-AUC is chosen over ROC-AUC as the primary performance metric.

$$\text{PR-AUC} = \sum_{i=1}^{n} (R_i - R_{i-1}) P_i \tag{3.3}$$

where P_i is the precision at threshold i, derived from:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{3.4}$$

and R_i is the recall at threshold i, derived from:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{3.5}$$

F2-Score measures the *F-beta* at $\beta = 2$, weighting recall twice as heavily as precision. This evaluation metric is important for vulnerability detection because a false negative or a missed vulnerability is much more dangerous than a false positive or a non-vulnerable that were flagged as vulnerable.

$$\text{F2-Score} = \frac{5 \times \text{Precision} \times \text{Recall}}{(4 \times \text{Precision}) + \text{Recall}} \tag{3.6}$$

MCC or Matthews Correlation Coefficient is a correlation coefficient, which uses all four cells of the confusion matrix. This evaluation metric is necessary for imbalanced binary classification. It complements PR-AUC by capturing TP, TN, FP, FN simultaneously, avoiding the PR-AUC blind spot of ignoring true negatives.

$$\text{MCC} = \frac{(\text{TP} \times \text{TN}) - (\text{FP} \times \text{FN})}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}} \tag{3.7}$$

3.8.2 Efficiency Metrics

Inference latency is the mean time, in milliseconds, needed to predict a single sample from the test set. The XGBoost is measured on a single CPU thread as its

computational cost on this dataset is relatively low and a fixed, single-threaded configuration allows retraining time and other efficiency metrics to be measured more consistently than on a GPU. In contrast, TabM and FT-Transformer were trained on a GPU for faster processing since training and hyperparameter tuning these models on a CPU would substantially increase computation time. Take note that the models were not evaluated on equivalent hardware, though it does not affect the predictive performance of all three models.

Retraining time is the wall-clock time, in minutes, needed to retrain the model from scratch on the training set. In a rapid CI/CD workflow, models may be retrained on every code push or at regular intervals. A model with a long retraining time creates a bottleneck in the pipeline, even if its inference is fast.

Model size is the storage footprint of the trained model, measured in megabytes (MB). It determines how much space is needed in CI/CD artifact registries and how costly it is to store, version, and transfer the model. This matters most for larger architectures such as FT-Transformer, whose parameter count tends to produce a considerably bigger model than tree-based methods.

Peak memory (RAM) is the maximum RAM used during inference, measured in megabytes (MB). It indicates whether a model can run within the resource limits of a standard CI runner, which are typically restricted in memory. Models with a heavy memory footprint, such as FT-Transformer, may be difficult to run on lightweight runners.

3.9 Hardware and Software Environment

All experiments are conducted in a consistent computational environment to ensure reproducibility. The primary software stack consists of Python 3.12.4 as the programming language, PyTorch 2.5.1 as the deep learning framework for the TabM and FT-Transformer implementations, the XGBoost library, Scikit-learn for preprocessing utilities, Optuna for hyperparameter optimization, and Pandas and NumPy for data manipulation.

The hardware environment used for training consists of a workstation equipped with an NVIDIA GeForce RTX 4060 GPU with 8GB VRAM, 16 GB of system RAM, and an Intel Core i7-14650HX processor. All experiments are run with a fixed global random seed (seed = 42) applied to all relevant libraries (NumPy, PyTorch, Python random module) to guarantee reproducibility of results.

CHAPTER IV

IMPLEMENTATION

4.1 Data Pipeline Implementation

The data pipeline is executed once per dataset by using `python-msrc.run_preprocessing`. Preprocessing, splitting, normalization, and class-weight computation live in separate modules and are then run by using `src/run_preprocessing.py`. The three model runners never access the raw CSV files. Instead, they read the processed split files and the `metadata.json` saved in `data/processed/ferenc/` and `data/processed/viszkok/`. This integration keeps the feature space identical across all model runners.

4.1.1 Data Acquisition

The unprocessed tabular data are placed inside `data/raw/Ferenc.csv` and `data/raw/Viszkok.csv`. `src/data_loading.py` reads each file using pandas and defines `load_ferenc()` and `load_viszkok()` functions. The binary target column is `Vuln`, as stated in both source datasets ((Ferenc et al., 2019); (Viszkok et al., 2021)). Paths, global seed, and split ratios are defined in `src/config.py` therefore later stages cannot drift from the original configuration defined in Section 3.4.

4.1.2 Preprocessing

The preprocessing step starts with feature filtering, implemented in `src/preprocessing.py` by using `filter_features()`. It applies the same rules defined in Section 3.3 to both datasets, using whichever columns are present. Identifier columns listed in `config.IDENTIFIER_COLUMNS` are removed, positional columns listed in `config.POSITIONAL_COLUMNS` are copied aside and are excluded from training, numeric columns with at most one unique value are dropped, and if two remaining numeric columns are almost the same (Pearson correlation ≥ 0.999), the earlier column is kept and the later one is dropped. A correlation threshold of 0.999 was selected to ensure that only the most highly correlated feature pairs are handled. A `PreprocessingReport` records the dropped columns and serializes them into `metadata.json`. In the previous sections, it is mostly stated

that there are 35 static metrics for Ferenc et al. (2019) dataset and 61 static metrics for Viszkok et al. (2021) dataset. However, after filtering with `filter_features()`, these counts are reduced to 32 features for Ferenc and 54 features for Viszkok. Tables 4.1 and 4.2 list the details of the removed columns, including the reason they are removed.

Table 4.1: Column removals details for Ferenc et al. (2019) dataset

Step	**Columns removed**	**Reason**
Identifiers	name, longname, path, full_ repo_path	Metadata; no metric signal
Zero variance	CLLC	Constant column; no information
Collinearity (abs $r \geq 0.999$)	HBUGS (kept HVOL), HTIME (kept HEFF)	Near-duplicate Halstead identities
Positional isolation	line, column, endline, endcolumn	Location artifact of dataset construction

Table 4.2: Column removals details for Viszkok et al. (2021) dataset

Step	**Columns removed**	**Reason**
Identifiers	name, longname, path, full_ repo_path, hash, type	Metadata; type is a constant label
Zero variance	WarningBlocker, WarningCritical, WarningInfo, WarningMinor	Constant columns; no information
Collinearity (abs $r \geq 0.999$)	CLLC (kept CLC), HVOL (kept HBUGS), HTIME (kept HEFF), HON_T (kept HLEN), HVOC (kept HON_ D)	Near-duplicate or derived pairs
Positional isolation	line, column, endline, endcolumn, start_line, start_ column, end_line, end_ column	Location artifact of dataset construction

Normalization is applied only after the split in order to make sure

the test distribution can not enter the transformer. `fit_quantile_transformer()` constructs a scikit-learn QuantileTransformer with `output_distribution="normal"`, `n_quantiles = min(1000, n_train)`, and `random_state = 42`, then calls fit on the training matrix alone. `apply_quantile_transform()` transforms train, validation, and test with that fitted object. The fitted transformer is saved to `quantile_transformer.joblib`.

```
# src/preprocessing.py -- train-only QuantileTransformer
n_quantiles = min(1000, len(X_train))
qt = QuantileTransformer(
    n_quantiles=n_quantiles,
    output_distribution="normal",
    random_state=random_state,
)
qt.fit(X_train)
return qt
```

Figure 4.1: Train-only QuantileTransformer fitting in src/preprocessing.py.

4.1.3 Data Split

The 60/20/20 stratified partition is implemented in `src/splitting.py`. Since scikit-learn's `train_test_split` only able to cut a matrix in two, the `train_test_split` has to be called twice and sequentially. The first stratify splits the data into training data and the remainder (40%). The remainder is then split into two sets, validation set and test set. Both calls use the stratify method on the current label vector and `random_state = 42`. Table 4.3 reports the actual numbers of data that are also reported inside `metadata.json`. The Viszkok training split contains 4,822 rows, rather than the rounded 4,823 shown in Table 3.2. This difference exists because of rounding in Table 3.2 and not a change of method.

```
# src/splitting.py -- two-step stratified 60/20/20 split
X_train, X_rest, y_train, y_rest = train_test_split(
    X, y,
    test_size=val_test_size,
    stratify=y,
    random_state=random_state,
)
relative_test_size = test_size / val_test_size
X_val, X_test, y_val, y_test = train_test_split(
    X_rest, y_rest,
    test_size=relative_test_size,
    stratify=y_rest,
    random_state=random_state,
)
```

Figure 4.2: Two-step stratified 60/20/20 split in src/splitting.py.

Table 4.3: Realized stratified split sizes (seed = 42)

Dataset	**Train (60%)**	**Validation (20%)**	**Test (20%)**	**Features after filtering**
Ferenc et al. (2019)	7,275	2,425	2,425	32
Viszkok et al. (2021)	4,822	1,608	1,608	54

4.1.4 Class Imbalance Handling

The imbalance of both datasets is handled using class weight. The weights are computed only from the training set in `src/class_weights.py`. `compute_class_weights()` returns one weight for each class, while `compute_scale_pos_weight()` returns a single ratio of non-vulnerable to vulnerable training rows. The resulting class weights are 0.5704 (non-vulnerable) and 4.0506 (vulnerable) for Ferenc, and 0.5678 (non-vulnerable) and 4.1857 (vulnerable) for Viszkok. The scaled weight ratios are 7.1013 for Ferenc and 7.3715 for Viszkok. The scaled ratio is needed because XGBoost expects `scale_pos_weight` while PyTorch expects

pos_weight. Both values are then stored in metadata.json and later read by the model runners.

```python
# src/class_weights.py
def compute_class_weights(y_train, num_classes=2):
    n_total = len(y_train)
    counts = y_train.value_counts().to_dict()
    weights = {}
    for class_label, n_class in counts.items():
        weights[int(class_label)] = n_total / (num_classes *
            n_class)
    return weights

def compute_scale_pos_weight(y_train):
    n_pos = int(y_train.sum())
    n_neg = len(y_train) - n_pos
    return n_neg / n_pos
```

Figure 4.3: Class-weight computation from Equation 3.1 and the XGBoost ratio in src/class_weights.py.

In summary, the preprocessing method produces these outputs: six split files, the positional columns file, the fitted QuantileTransformer, and metadata.json. The preprocessing steps are run only at the beginning and are never repeated during later training or testing. To avoid data leakage, the data are split before normalization and class-weight computation. Normalization is then fit on the training set only, and class weights are also computed from the training labels only. Therefore, the full order of the preprocessing is: filter features, split into train/validation/test, QuantileTransformer normalization, then compute class weights.

```
# src/run_preprocessing.py -- split before normalization
X_train, X_val, X_test, y_train, y_val, y_test = (
    stratified_train_val_test_split(X, y)
)
qt = fit_quantile_transformer(X_train)
X_train_t, X_val_t, X_test_t = apply_quantile_transform(
    qt, X_train, X_val, X_test
)
class_weights = compute_class_weights(y_train)
```

Figure 4.4: Preprocessing orchestration that splits before normalization in src/run_preprocessing.py.

4.2 Model & Training Pipeline Implementation

Each model has its own class, but all three use the same method names and the same order: `tune()`, `train()`, then `predict_proba()`. The code inside each method is customized for that model. The runners also follow the same structure, with model-specific details inside. Their shared order is: load the same preprocessed data, read `scale_pos_weight` from `metadata.json`, run the sequence above, then save the model checkpoint and write the results to a JSON file for that model and dataset.

4.2.1 XGBoost

XGBoost is not reimplemented, instead it is adopted from the `xgboost` library (Chen and Guestrin, 2016). The `XGBoostVulnModel` class in `src/models/xgboost_model.py` wraps the imported model `xgb.XGBClassifier` along with its customization. The fixed settings are set inside `_base_params()`, which are: `objective="binary:logistic"`, `eval_metric="aucpr"`, the stored `scale_pos_weight`, `n_jobs=1` is chosen so that later CPU latency is comparable to the PyTorch models, and `random_state=42`. The search space used by `tune()` then follows the setup defined in Table 3.3. After `tune()` stores the optimal parameters, `train()` then builds a new classifier, calls `fit` with the validation split as `eval_set` and 50 early-stopping rounds, and returns the wall-clock duration of that fit in minutes. The fitted model is saved using joblib to

models/<dataset>/xgboost.joblib.

4.2.2 TabM

TabM is reimplemented in PyTorch in src/models/tabm_model.py, following Gorishniy et al. (2025) rather than using the tabm library. TabMVulnModel wraps the custom TabMNet architecture. The fixed settings for TabM are the pos_weight stored in the dataset metadata (used in nn.BCEWithLogitsLoss), seed=42, and CUDA for training whenever torch.cuda.is_available() is true. In the architecture, TabMNet first applies a piecewise-linear numerical embedding (NumericalEmbedding), which encodes each feature with 32 quantile-based bins fitted on X_train and stored as a model buffer, and projects it to a d_embedding-dimensional vector. The per-feature vectors are concatenated and passed through stacked TabMEnsembleLayer blocks that apply Equation 3.2, each followed by ReLU and Dropout for every ensemble member. During training, each mini-batch is split into k disjoint chunks so that each of the k members sees different objects, while inference (and the optional shared_batch=True mode) replicates the embedded input k times. Each member then has its own non-shared linear head that produces one logit, supervised with binary cross-entropy against the label of its own object. At inference, a sigmoid is applied to each logit and the k probabilities are averaged, then thresholded at 0.5. train() rebuilds the network from the architecture configuration, runs the Adam training loop for up to 300 epochs with early stopping on validation PR-AUC (patience of 20 epochs), and returns the wall-clock duration in minutes. The results are saved to models/<dataset>/tabm.pt (state dict, including the fitted bin edges) and tabm_arch.json.

```python
# src/models/tabm_model.py -- TabMEnsembleLayer.forward
def forward(self, x: torch.Tensor) -> torch.Tensor:
    # x may be (batch, n) or (batch, k, n); the embedding is applied per
    # object, so all leading dimensions are flattened first.
    leading = x.shape[:-1]
    flat    = x.reshape(-1, self.n_features)            # (M, n_features)
    enc     = self._piecewise_linear(flat)              # (M, n, n_bins)
    # (M, n, n_bins) x (n, d, n_bins) -> (M, n, d)
    emb = torch.einsum("mnk,ndk->mnd", enc, self.weight) + self.bias
    return emb.reshape(*leading, self.out_dim)
```

Figure 4.5: TabM ensemble-layer forward pass implementing Equation 3.2 in src/models/tabm_model.py.

4.2.3 FT-Transformer

FT-Transformer is also reimplemented using PyTorch in `src/models/ft_transformer.py`, following Gorishniy et al. (2021). `FTTransformerVulnModel` wraps the custom FT-Transformer network. The fixed settings are the stored `pos_weight` from `metadata.json` (used in `BCEWithLogitsLoss`), `seed=42`, and CUDA when `torch.cuda.is_available()` is true. In the architecture, `FeatureTokenizer` converts each numeric feature into a token of width `d_token`. Then, a learnable `[CLS]` token is prepended. After that, an `FTTransformerLayer` stacks PreNorm multi-head self-attention and a feed-forward block. Finally, a linear head converts the `[CLS]` representation to a single logit. The search space used by `tune()` that follows the defined set in Table 3.3. After `tune()` stores `best_params`, `train()` rebuilds the network, runs the training loop with early stopping of 20 epochs, and returns the wall-clock duration in minutes. The results are saved to two different files, `models/<dataset>/ft_transformer.pt` for the state dict and `ft_transformer_arch.json`.

4.3 Tuning & Training-loop Implementation

The hyperparameter tuning is implemented using Optuna 4.1.0 with the Tree-structured Parzen Estimator (Akiba et al., 2019). The trial counts, early-stopping patience, and search ranges follow Table 3.3. In every trial, the model is scored by validation PR-AUC, and Optuna searches for hyperparameters that maximize that score. The `tune()` method receives only the training and validation data. After the search, each model is retrained from scratch with the best hyperparameters. The total Optuna search time is not stored. Instead, only the duration of this final retrain is recorded as the retraining time.

4.3.1 XGBoost Search

`XGBoostVulnModel.tune()` starts an Optuna search with `TPESampler(seed=42)` and `direction="maximize"`, then runs 50 trials. On each trials, Optuna uses the ranges defined in Table 3.3, builds an `XGBClassifier`, and fits it with 50-round early stopping on the validation set. Each trial is scored by validation PR-AUC using `average_precision_score`. This XGBoost search does not use a MedianPruner since it trains faster than the deep learning models. Note that this choice may improve the performance of XGBoost. After tuning finishes, `train()` runs one final fit with the best parameters and records that fit time as the retraining duration.

4.3.2 TabM and FT-Transformer Search

Both TabM and FT-Transformer use the same tuning loop. In `tune()`, Optuna is set up with a MedianPruner which contains 5 startup trials and 10 warmup steps, then it runs 50 trials. Inside each trial, `_training_loop` builds the network, trains it with Adam using the sampled weights and weight decay, and runs for up to 150 epochs. After every epoch, validation PR-AUC is computed and reported to Optuna, which may stop a weak trial early using `TrialPruned`. In addition, training also stops if validation PR-AUC does not improve for 20 epochs. Before the trial returns its score, the best weights from that trial are restored. For FT-Transformer, `d_token` is kept as a multiple of `n_heads` so `nn.MultiheadAttention` can run. In conclusion, TPE chooses the hyperparameters, MedianPruner stops weak trials, and patience 20 provides early stopping as stated in Table 3.3.

```python
# src/models/tabm_model.py -- Optuna study and pruned training loop
sampler = optuna.samplers.TPESampler(seed=self.seed)
pruner  = optuna.pruners.MedianPruner(
    n_startup_trials=5, n_warmup_steps=10
)
study = optuna.create_study(
    direction="maximize", sampler=sampler, pruner=pruner
)
study.optimize(objective, n_trials=n_trials)

# inside _training_loop, after each epoch:
if trial is not None:
    trial.report(val_auc, epoch)
    if trial.should_prune():
        raise optuna.exceptions.TrialPruned()
if no_improve >= patience:
    break
```

Figure 4.6: Optuna TPE study with MedianPruner and early stopping in src/models/tabm_model.py.

4.3.3 Final Retrain

After `best_params` is stored, `TabMVulnModel.train()` and the FT-Transformer `train()` method use the same training loop again, but with `max_epochs=300` and no Optuna trial object, therefore pruning is turned off. The time returned by this final training is saved as `retraining_time_min` in the metrics JSON. For XGBoost, `train()` also measures only this final fit, not only the 100 tuning fits. The runner then saves the model results and continues to evaluation.

4.4 Evaluation and Comparison Implementation

Evaluation is implemented in `src/evaluate.py` and used by all three runners. It is called once per model after the final retrain. This section explains how each model summarized its results and how the comparison table is produced,

including the two time columns. The held-out test split is used here for the first time inside each runner.

4.4.1 Performance Fields

`predict_proba` returns the positive-class probability for every test row. `compute_performance_metrics` passes those probabilities to scikit-learn `average_precision_score` to obtain the PR-AUC. It then thresholds at 0.5 and computes F2-Score with `beta=2` and MCC using `matthews_corrcoef`. The three numbers are written inside the metrics JSON under the performance section.

```
# src/evaluate.py -- test-set performance fields
y_pred = (y_prob >= threshold).astype(int)
pr_auc = average_precision_score(y_true, y_prob)
f2     = fbeta_score(y_true, y_pred, beta=2, zero_division=0)
mcc    = matthews_corrcoef(y_true, y_pred)
return {"pr_auc": float(pr_auc), "f2_score": float(f2), "mcc"
   : float(mcc)}
```

Figure 4.7: Test-set PR-AUC, F2-Score, and MCC computation in src/evaluate.py.

4.4.2 Efficiency Fields including Training Time and Testing Time

The comparison table has two time columns and they measure different operations. Testing time measures how long an already trained model takes to score new data as inference latency. `measure_inference_latency_ms` calls `predict_proba` on the test set, discards the first warm-up call, times the next 10 calls, converts each duration to milliseconds per sample, and stores the mean as `inference_latency_ms_per_sample`. TabM and FT-Transformer run this on a GPU, while XGBoost already uses `n_jobs=1` for the single-thread CPU. This follows the CI/CD-oriented protocol in Section 3.8.2.

The training time inside the table is only retraining time. Each model's `train()` times one final fit with the best hyperparameters and saves that value as `retraining_time_min`. This shows how long a refit takes when the training set is updated, and it does not include the 50 Optuna trials. The remaining efficiency fields are file size and peak RAM usage.

```
# src/evaluate.py -- per-sample CPU inference latency
_ = predict_fn(X)  # warm-up, not timed
latencies = []
for _ in range(n_repeats):
    t0 = time.perf_counter()
    _ = predict_fn(X)
    latencies.append((time.perf_counter() - t0) * 1000 / len(
        X))
return float(np.mean(latencies))
```

Figure 4.8: CPU per-sample inference latency measurement in src/evaluate.py.

`get_model_size_mb` returns the model size in megabytes. `src/compare.py` prefers the on-disk file under `models/` when assembling the table. `measure_peak_ram_mb` polls process RSS every millisecond during one `predict_proba` call and stores the maximum as `peak_ram_inference_mb`.

4.4.3 How the Comparison Table is Generated

After all six runners have finished, `python -m src.compare` loads `results/metrics/<model>_<dataset>.json` for XGBoost, TabM, and FT-Transformer on both datasets. `build_rows()` copies the performance and efficiency values into one row per model. The script then prints a console table and writes `comparison_table.json` and `comparison_table.md`. Each column comes from a runner JSON, and `compare.py` does not retrain or re-score the test set. These numeric results are used as discussion material in Chapter 5.

```
# src/compare.py -- mapping JSON fields into table columns
rows.append({
    "model": MODEL_LABELS[model],
    "pr_auc": float(perf["pr_auc"]),
    "f2_score": float(perf["f2_score"]),
    "mcc": float(perf["mcc"]),
    "inference_latency_ms_per_sample":
        float(eff["inference_latency_ms_per_sample"]),
    "retraining_time_min": float(eff["retraining_time_min"]),
    "model_size_mb": float(eff["model_size_mb"]),
    "peak_ram_inference_mb": float(eff["peak_ram_inference_mb
        "]),
})
```

Figure 4.9: Mapping of stored JSON fields into comparison-table columns in src/compare.py.

CHAPTER V

RESULTS AND DISCUSSION

5.1 Experimental Design

Table 5.1 records the final inputs that enter the model training after preprocessing. The identifier columns, zero-variance columns, and one member of each near-perfectly collinear pair are removed from both datasets. In addition, the positional columns are isolated so that they never enter the feature matrix. This leaves 32 usable features on Ferenc and 54 usable features on Viszkok. The cleaned datasets are then splitted using stratify with a 60/20/20 partition, which yields the exact number of train, validation, and test sizes. Finally, the `scale_pos_weight` is computed as the ratio of non-vulnerable to vulnerable rows and this is a fixed ratio and is not tuned across all models.

Table 5.1: Realized feature counts, stratified split sizes (seed = 42), and class-weight ratios

Item	**Ferenc et al. (2019)**	**Viszkok et al. (2021)**
Features after filtering	32	54
Train (60%)	7,275	4,822
Validation (20%)	2,425	1,608
Test (20%)	2,425	1,608
scale_pos_weight	7.1013	7.3715

5.2 Experimental Results

5.2.1 Best Hyperparameters

This subsection reports the optimal configurations found on each model within our search space. Each tuning was run independently on each dataset, therefore the tuning values differs between Ferenc and Viszkok dataset, even though they both used the same search settings.

On XGBoost, the winning Ferenc trial prefers less trees, a deeper, more heavily subsampled ensemble (780 trees, depth 12, subsample 0.5132), while the trial on Viszkok dataset prefers fewer, twice as many trees with a higher learning rate and

subsample close to one. The final best hyperparameters for XGBoost are summarized in Table 5.2.

Table 5.2: Best XGBoost hyperparameters after 50 Optuna TPE trials on each validation set

Parameter	**Ferenc**	**Viszkok**
n_estimators	780	1448
max_depth	12	7
learning_rate	0.065	0.1095
subsample	0.799	0.9395
colsample_bytree	0.578	0.7700
min_child_weight	2	4
reg_alpha	0.29	0.3595
reg_lambda	4.33	3.1062
Best validation PR-AUC	0.81	0.9027

On TabM, the winning Ferenc trial is a wider network with a smaller ensemble (three layers, hidden width 366, $k = 43$), whereas Viszkok selects a deeper, narrower network with a slightly larger ensemble (three layers, hidden width 196, $k = 46$). Batch size is 1024 in both winning trials. The final best hyperparameters for TabM are summarized in Table 5.3.

Table 5.3: Best TabM hyperparameters after 50 Optuna TPE trials on each validation set

Parameter	**Ferenc**	**Viszkok**
num_layers	3	3
hidden_dim	366	196
dropout	0.3857	0.3831
learning_rate	0.006	0.0098
weight_decay	8.749e-5	2.492e-6
batch_size	1024	1024
k	43	46
Best validation PR-AUC	0.8091	0.9035

On FT-Transformer, the winning Ferenc trial uses a wider token, more heads, and three layers (`d_token` = 336, eight heads, batch size 512), while Viszkok selects

a narrower token, fewer heads, and two layers (`d_token` = 268, four heads, batch size 256). The final best hyperparameters for FT-Transformer are summarized in Table 5.4.

Table 5.4: Best FT-Transformer hyperparameters after 50 Optuna TPE trials on each validation set

Parameter	**Ferenc**	**Viszkok**
n_heads	8	4
num_layers	3	2
attention_dropout	0.1302	0.1734
ffn_dropout	0.2651	0.1024
ffn_factor	1.7994	2.3851
learning_rate	3.124e-4	1.032e-4
weight_decay	9.387e-6	2.410e-5
batch_size	512	256
d_token	336	268
Best validation PR-AUC	0.7799	0.8924

5.2.2 Performance Results

After the final retrain with the hyperparameters in Tables 5.2–5.4, each model is scored once on the held-out test split. The fields that are reported are PR-AUC, F2-Score, and MCC as defined in Section 3.8.1.

On Ferenc, the three models receive the same 32-feature test matrix of 2,425 functions. PR-AUC is computed from the predicted probabilities, while F2 and MCC are computed from hard labels obtained by applying a threshold of 0.5. The test-set predictive scores for Ferenc et al. (2019) are summarized in Table 5.5.

Table 5.5: Test-set predictive performance on Ferenc et al. (2019), 32 features

Model	**PR-AUC**	**F2-Score**	**MCC**
XGBoost	0.8039	0.7071	0.6750
TabM	0.7658	0.7002	0.6367
FT-Transformer	0.7518	0.6714	0.4679

XGBoost leads every column on Ferenc. TabM is second on all three metrics and stays within 0.0381 PR-AUC of XGBoost, while FT-Transformer trails TabM

by a further 0.0140 PR-AUC. The spread however, is wider on MCC with 0.6750 against 0.6367 against 0.4679 than on PR-AUC, so the models separate more clearly once scores are collapsed at threshold 0.5.

On Viszkok, the same evaluation is applied to the 54-feature test matrix of 1,608 functions, again after an independent retrain with that dataset's winning hyperparameters. The test-set predictive scores for Viszkok et al. (2021) are summarized in Table 5.6.

Table 5.6: Test-set predictive performance on Viszkok et al. (2021), 54 features

Model	**PR-AUC**	**F2-Score**	**MCC**
XGBoost	0.9071	0.8091	0.7773
TabM	0.8816	0.7930	0.6826
FT-Transformer	0.8856	0.8193	0.7298

XGBoost again leads PR-AUC and MCC. FT-Transformer is second on those two fields and only 0.0040 PR-AUC above TabM, so the deep models are nearly tied on the primary metric, PR-AUC. FT-Transformer also records the highest F2 (0.8193), slightly above XGBoost (0.8091) and TabM (0.7930). That higher F2 on FT-Transformer is the only time a deep learning model wins on either dataset.

Across both datasets, XGBoost performs the best, scoring highest on the primary metric (PR-AUC) and on MCC. Its PR-AUC lead over the best deep model is 0.0381 on Ferenc (against TabM) and 0.0215 on Viszkok (against FT-Transformer), so its advantage is clearer on the static-metric table than on the process-enriched one. Between the deep models the ranking differs depending on the dataset. TabM is ahead of FT-Transformer on Ferenc, while FT-Transformer is ahead on Viszkok, narrowly on PR-AUC but more clearly on F2 and MCC. This suggest that TabM performs better on a dataset that has more rows while FT-Transformer performs better on a dataset that has more features. FT-Transformer's F2 on Viszkok is also the only metric on which a deep model beats XGBoost on either dataset. The absolute scores are higher for every architecture on Viszkok than on Ferenc, which is consistent with the added process metrics, however, the two datasets are not ranked against each other as a single scoreboard.

5.2.3 Efficiency Results

The efficiency of each model are measured from the same final retrain used for section 5.3.2. There are four metrics to be compared between each model: inference latency, retraining time, model size, and peak RAM at inference. Optuna search time is excluded because it is an amortized model selection cost. Retraining time is considered as the primary cost since it is the recurring cost of refitting the model after the hyperparameters are already fixed.

Table 5.7: Test-set efficiency on Ferenc et al. (2019), 32 features

Model	**Latency (ms/sample)**	**Retrain (min)**	**Size (MB)**	**Peak RAM (MB)**
XGBoost	0.011	0.021	1.851	291
TabM	0.987	0.294	2.349	1,401
FT-Transformer	1.886	1.880	10.389	2,021

XGBoost is the cheapest model on Ferenc across every metric: 0.021 min to retrain, 0.011 ms/sample to score, a 1.851 MB checkpoint, and 291 MB peak RAM. TabM sits in the middle, requiring 0.294 min to retrain (about 14 times XGBoost) and 0.987 ms/sample latency (about 90 times XGBoost), with a 2.349 MB checkpoint and 1,401 MB peak RAM. FT-Transformer is the most expensive model by a wide margin: 1.880 min to retrain (about 6.4 times TabM and 90 times XGBoost), 1.886 ms/sample latency, a 10.389 MB checkpoint, and 2,021 MB peak RAM.

Table 5.8: Test-set efficiency on Viszkok et al. (2021), 54 features

Model	**Latency (ms/sample)**	**Retrain (min)**	**Size (MB)**	**Peak RAM (MB)**
XGBoost	0.003	0.008	0.409	288
TabM	0.998	0.061	1.037	1,705
FT-Transformer	1.634	0.764	5.228	2,472

XGBoost again dominates the Viszkok efficiency table across every metric: 0.008 min to retrain, 0.003 ms/sample latency, the smallest checkpoint (0.409 MB), and 288 MB peak RAM. TabM sits in the middle, requiring 0.061 min to retrain (about 7.6 times XGBoost) and 0.998 ms/sample latency (about 333 times XGBoost), with a 1.037 MB checkpoint and 1,705 MB peak RAM. FT-Transformer is the most

expensive model: 0.764 min to retrain (about 12.5 times TabM and 95.5 times XGBoost), 1.634 ms/sample latency, a 5.228 MB checkpoint, and the highest peak RAM at 2,472 MB.

In summary, the efficiency order across both datasets from most efficient to least efficient is XGBoost, then TabM, then FT-Transformer. When prioritizing solely on efficiency in a rapid CI/CD environment, XGBoost is the main choice for the fastest and cheapest option. For deep learning options, TabM is the stronger option because it is cheaper than FT-Transformer on retrain, latency, size, and peak RAM.

5.3 Discussion

5.3.1 Predictive Performance

XGBoost is the strongest model overall, ranking first on PR-AUC and MCC on both datasets, with PR-AUC leads of 0.0381 on Ferenc and 0.0215 on Viszkok over the best deep model. The only metric on which a deep model beats it is F2 on Viszkok, where FT-Transformer scores 0.8193 against 0.8091, a margin of only 0.0102. This result should be read with care because F2 weights recall twice as heavily as precision and is computed from hard labels at a fixed threshold of 0.5, whereas PR-AUC evaluates the full ranking of predicted probabilities. FT-Transformer is lower than XGBoost on both PR-AUC (0.8856 against 0.9071) and MCC (0.7298 against 0.7773), so it does not rank functions better. The precision and recall values explain why it still wins on F2. At the 0.5 threshold, FT-Transformer reaches a recall of 0.8646 against 0.8125 for XGBoost, but its precision is only 0.6776 against 0.7959. Since F2 weights recall twice as heavily as precision, the recall gain outweighs the precision loss, whereas MCC, which balances both, favors XGBoost. In summary, FT-Transformer flags more functions as vulnerable, catching roughly ten more true positives at the cost of roughly forty more false alarms. The margin is small and comes from a single seed and test split, so it reflects a different operating point at a fixed threshold more than a better detector.

Between the two deep models, the choice depends on the dataset. On Ferenc, which has more rows (12,125 functions) but fewer features (32), TabM is ahead of FT-Transformer on every metric, and the gap is largest on MCC (0.6367 against 0.4679). On Viszkok, which has fewer rows (8,038 functions) but more features (54) because of the added process metrics, FT-Transformer is ahead of TabM, narrowly on PR-AUC (0.8856 against 0.8816) and more clearly on F2 and MCC. This pattern

suggests that TabM is better suited to datasets with more rows, while FT-Transformer benefits more from richer feature sets, possibly because its attention layers can model interactions among many feature tokens. Therefore, FT-Transformer is only justified when the dataset contains rich features such as process metrics, and TabM is the more practical default among the deep models in our set.

5.3.2 Computational Efficiency

XGBoost is the clear winner on inference latency at 0.011 ms/sample on Ferenc and 0.003 ms/sample on Viszkok, compared with roughly 1 to 1.9 ms/sample for TabM and FT-Transformer. For a single function the difference is imperceptible, but it grows with the number of functions scored: 10,000 functions take about 0.11 s with XGBoost on Ferenc, against about 19 s with FT-Transformer. If the system rechecks code every time the developer pauses typing, or repeatedly within a short time, XGBoost is the practical choice because the checks run continuously and add up. If checks run only on demand, such as on a button press or at occasional intervals, the deep models are still acceptable since the cost per check is only a few milliseconds.

The same reasoning applies to retraining, but the gaps are larger. XGBoost retrains in 0.021 min on Ferenc and 0.008 min on Viszkok, TabM needs 0.294 and 0.061 min, and FT-Transformer needs 1.880 and 0.764 min. If the model is refreshed only once every hour or two, or a couple of times a day, even the 1.9 minutes of FT-Transformer is acceptable. If the model is retrained on every push in a busy CI/CD pipeline, those minutes accumulate into queue delays and compute cost, so XGBoost is the better fit. These figures exclude the Optuna search time, so the full cost of introducing a new model type would be higher for the deep models.

All three models are relatively small, with checkpoints between 0.409 MB (XGBoost on Viszkok) and 10.389 MB (FT-Transformer on Ferenc), so storage is not a limiting factor on any modern device or server. The deep models are nonetheless larger than XGBoost, and FT-Transformer is the largest on both datasets. The difference is likely to matter little here, but it could grow if the training data became larger or the architectures had to be scaled up, so model size should be rechecked in that case.

Memory is the metric where the gap between model families is most practically relevant. XGBoost stays below 300 MB on both datasets (291 and 288 MB), so it can run alongside other processes without noticeable impact. TabM and FT-

Transformer exceed 1 GB, reaching 1,401 and 2,021 MB on Ferenc and 1,705 and 2,472 MB on Viszkok. In a shared or busy environment such as a developer machine or a CI runner, using these models alongside with other applications may degrade their efficiency. Memory is therefore the main practical argument against deploying the deep models in resource-constrained settings.

5.4 Comparison With Prior Works

Ferenc et al. (2019) and Viszkok et al. (2021) evaluated several classifiers on the same two datasets using Precision, Recall, and F1-score. A direct comparison with this study is not fair, because the experimental environment differs. Their models were trained on an 80/10/10 split, with hyperparameters tuned by grid search and 10-fold cross-validation, ranked by F-measure, and with imbalance handled by random over-sampling and under-sampling. This study uses a different split and a different imbalance handling strategy (Sections 3.7, 3.8, and 5.1). Our hyperparameter tuning also targets PR-AUC, with F2 and MCC as supporting metrics, to handle the class imbalance of the datasets, not Precision, Recall, or F1. Nevertheless, the comparison is presented in Tables 5.9 and 5.10 for reference. It covers only the classifiers from the original studies, since other studies use varied environments that would be much harder to compare. For this section only, Precision, Recall, and F1 for our models are computed from the same held-out test predictions at threshold 0.5, which does not change the primary metrics.

Table 5.9: Precision, Recall, and F1-score on the Ferenc static-metric table: published Ferenc et al. (2019) scores versus this study's held-out test scores at threshold 0.5

Model	**Precision**	**Recall**	**F1-score**
KNN	0.909	0.659	0.764
DT	0.737	0.697	0.716
RFC / Forest	0.931	0.578	0.713
SDNN	0.873	0.600	0.711
CDNN	0.911	0.579	0.708
SVM	0.953	0.514	0.668
LogReg	0.753	0.212	0.331
LinReg	0.843	0.154	0.261
NB	0.224	0.117	0.153
XGBoost (this study)	0.711	0.709	0.710
TabM (this study)	0.538	0.736	0.621
FT-Transformer (this study)	0.368	0.846	0.513

Table 5.10: Precision, Recall, and F1-score on the Viszkok static-plus-process table: published Viszkok et al. (2021) F-measure-optimal scores versus this study's held-out test scores at threshold 0.5

Model	**Precision**	**Recall**	**F1-score**
RFC	0.958	0.760	0.848
DT	0.909	0.753	0.824
KNN	0.949	0.713	0.814
SDNN	0.921	0.716	0.805
CDNN	0.928	0.706	0.802
SVM	0.861	0.721	0.785
LogReg	0.743	0.517	0.609
LinReg	0.540	0.594	0.566
NB	0.278	0.120	0.167
XGBoost (this study)	0.796	0.812	0.804
TabM (this study)	0.678	0.844	0.752
FT-Transformer (this study)	0.678	0.865	0.760

Our models score lower than the best classifiers from the previous studies on

these metrics. As noted earlier, this is partly because the comparison is not direct, and partly because our study did not optimize the metrics used in the prior work. For a fair comparison, future work needs an identical environment in which all previous models and our models are compared, with hyperparameters tuned on the same set of metrics.

5.5 Limitations of the Finding

Several limitations of this study come from its own design and scope. The experimental environment was not fully identical across models: XGBoost was run on a CPU while TabM and FT-Transformer were run on a GPU, which may affect the efficiency measurements, such as latency, retraining time, and peak RAM, and so limits how strictly the models can be compared on those metrics. The comparison with prior work in Section 5.5 is also limited because the previous studies used a different environment. A fairer comparison would need to reimplement every model, including those from the original studies, under one balanced setup. This was not done because the scope of this study was limited to reimplementing the three selected models. All results also come from a single random seed (42), so the reported differences, especially the small ones such as the F2 gap on Viszkok, may be partly due to randomness, and multiple seeds would be needed to average this out. In addition, most parameters that were not stated or implied by the study design were kept at neutral choices or at the original setup of the previous studies and were not tuned, such as the maximum of 150 epochs during hyperparameter tuning and 300 epochs for the final training of TabM and FT-Transformer. These untuned settings may favor some models over others. Finally, hyperparameter tuning used a single fixed validation split instead of cross-validation. Cross-validation would give a more stable estimate of each trial's quality, but it was not used because of limited resources and time.

Other limitations come from the data, which are outside the control of this study. The datasets are relatively small for deep learning models, although they are sufficient for classical machine learning models such as XGBoost. This may have worked against TabM and FT-Transformer, so their results here should not be taken as their performance on larger datasets. The datasets are also severely imbalanced, with about seven non-vulnerable functions for every vulnerable one. This makes the minority class harder to learn and makes the threshold-dependent metrics more sensitive to a small number of predictions. A dataset with a more balanced class dis-

tribution would be needed to examine how much of the observed behavior is caused by the imbalance.

REFERENCES

Afsah Anwar, Aminollah Khormali, Jinchun Choi, Hisham Alasmary, Sung Choi, Saeed Salem, DaeHun Nyang, and David Mohaisen. Measuring the Cost of Software Vulnerabilities. *ICST Transactions on Security and Safety*, 7(23):164551, June 2020. ISSN 2032-9393. doi: 10.4108/eai.13-7-2018.164551. URL http://eudl.eu/doi/10.4108/eai.13-7-2018.164551.

Foteini Cheirdari and George Karabatis. Analyzing False Positive Source Code Vulnerabilities Using Static Analysis Tools. In *2018 IEEE International Conference on Big Data (Big Data)*, pages 4782–4788, Seattle, WA, USA, December 2018. IEEE. ISBN 978-1-5386-5035-6. doi: 10.1109/BigData.2018.8622456. URL https://ieeexplore.ieee.org/document/8622456/.

Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, August 2016. doi: 10.1145/2939672.2939785. URL http://arxiv.org/abs/1603.02754. arXiv:1603.02754 [cs.LG].

Charles Elkan. The Foundations of Cost-Sensitive Learning.

Rudolf Ferenc, Peter Hegedus, Peter Gyimesi, Gabor Antal, Denes Ban, and Tibor Gyimothy. Challenging Machine Learning Algorithms in Predicting Vulnerable JavaScript Functions. In *2019 IEEE/ACM 7th International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering (RAISE)*, pages 8–14, Montreal, QC, Canada, May 2019. IEEE. ISBN 978-1-7281-2272-4. doi: 10.1109/RAISE.2019.00010. URL https://ieeexplore.ieee.org/document/8823747/.

Jerome H. Friedman. http://www.jstor.org Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. URL http://www.jstor.org/stable/2699986.

Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. Revisiting Deep Learning Models for Tabular Data, October 2023. URL http://arxiv.org/abs/2106.11959. arXiv:2106.11959 [cs].

Yury Gorishniy, Akim Kotelnikov, and Artem Babenko. TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling, February 2025. URL http://arxiv.org/abs/2410.24210. arXiv:2410.24210 [cs].

Jacob A. Harer, Louis Y. Kim, Rebecca L. Russell, Onur Ozdemir, Leonard R. Kosta, Akshay Rangamani, Lei H. Hamilton, Gabriel I. Centeno, Jonathan R. Key, Paul M. Ellingwood, Erik Antelman, Alan Mackay, Marc W. McConley, Jeffrey M. Opper, Peter Chin, and Tomo Lazovich. Automated software vulnerability detection with machine learning, August 2018. URL http://arxiv.org/abs/1803.04497. arXiv:1803.04497 [cs.SE].

Alessandro Ingrosso and Sebastian Goldt. Data-driven emergence of convolutional structure in neural networks. *Proceedings of the National Academy of Sciences*, 119(40):e2201854119, October 2022. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.2201854119. URL http://arxiv.org/abs/2202.00565. arXiv:2202.00565 [cond-mat.dis-nn].

Ilias Kalouptsoglou, Miltiadis Siavvas, Dionysios Kehagias, Alexandros Chatzigeorgiou, and Apostolos Ampatzoglou. Examining the Capacity of Text Mining and Software Metrics in Vulnerability Prediction. *Entropy*, 24(5):651, May 2022. ISSN 1099-4300. doi: 10.3390/e24050651. URL https://www.mdpi.com/1099-4300/24/5/651.

Balazs Mosolygo, Norbert Vandor, Gabor Antal, Peter Hegedus, and Rudolf Ferenc. Towards a Prototype Based Explainable JavaScript Vulnerability Prediction Model. In *2021 International Conference on Code Quality (ICCQ)*, pages 15–25, Moscow, Russia, March 2021. IEEE. ISBN 978-1-7281-8476-0. doi: 10.1109/ICCQ51190.2021.9392984. URL https://ieeexplore.ieee.org/document/9392984/.

Ziyue Pan, Wenbo Shen, Xingkai Wang, Yutian Yang, Rui Chang, Yao Liu, Chengwei Liu, Yang Liu, and Kui Ren. Ambush From All Sides: Understanding Security Threats in Open-Source Software CI/CD Pipelines. *IEEE Transactions on Dependable and Secure Computing*, 21(1):403–418, January 2024. ISSN 1545-5971, 1941-0018, 2160-9209. doi: 10.1109/TDSC.2023.3253572. URL https://ieeexplore.ieee.org/document/10061526/.

Jhansi Rani Prathuri, Richard Delwin Myloth, and Pavan Rajkumar Magesh. Software Change Risk Prediction Using Supervised Machine Learning and Feature Impact Analysis. In *2026 IEEE 5th World Conference on Applied Intelligence and Computing (AIC)*, pages 1417–1423, Jabalpur, India, August 2026. IEEE. ISBN 979-8-3195-1898-9. doi: 10.1109/AIC70896.2026.11717248. URL https://ieeexplore.ieee.org/document/11717248/.

Abdoulaye Sakho, Emmanuel Malherbe, and Erwan Scornet. Do we need rebalancing strategies? A theoretical and empirical study around SMOTE and its variants, March 2026. URL http://arxiv.org/abs/2402.03819. arXiv:2402.03819 [stat].

Hazem Peter Samoaa, Firas Bayram, Pasquale Salza, and Philipp Leitner. A systematic mapping study of source code representation for deep learning in software engineering. *IET Software*, 16(4):351–385, August 2022. ISSN 1751-8806. doi: 10.1049/sfw2.12064. URL https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2.12064.

Abdullah Sheneamer. Vulnerable JavaScript functions detection using stacking of convolutional neural networks. *PeerJ Computer Science*, 10:e1838, February 2024. ISSN 2376-5992. doi: 10.7717/peerj-cs.1838. URL https://peerj.com/articles/cs-1838.

Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90, May 2022. ISSN 15662535. doi: 10.1016/j.inffus.2021.11.011. URL https://linkinghub.elsevier.com/retrieve/pii/S1566253521002360.

Tamás Viszkok, Péter Hegedűs, and Rudolf Ferenc. Improving Vulnerability Prediction of JavaScript Functions Using Process Metrics, May 2021. URL http://arxiv.org/abs/2105.07527. arXiv:2105.07527 [cs].

Han-Jia Ye, Si-Yang Liu, Hao-Run Cai, Qi-Le Zhou, and De-Chuan Zhan. A Closer Look at Deep Learning Methods on Tabular Datasets, November 2025. URL http://arxiv.org/abs/2407.00956. arXiv:2407.00956 [cs].

APPENDIX A

ATTACHMENT

1.1 Ferenc Dataset Summary Statistic Table

Observations: 12,125 **Features:** 35
Vulnerable: 1,496 **Non-vulnerable:** 10,629

Table 1.1: Ferenc Dataset Summary Statistics

Metric	Mean	Std Dev	Min	Median	Max
Complexity					
McCC	3.071	4.789	1.000	2.000	137.0
NL	0.765	1.175	0.000	0.000	14.000
NLE	0.705	1.019	0.000	0.000	9.000
NUMPAR	1.161	1.260	0.000	1.000	13.000
CYCL	2.674	3.581	1.000	1.000	121.0
PARAMS	1.191	1.263	0.000	1.000	13.000
CYCL_DENS	67.212	51.633	0.000	50.000	900.0
Size / LOC					
LLOC	8.244	17.737	1.000	4.000	1071.0
LOC	10.669	28.641	1.000	4.000	1244.0
NOS	5.168	9.934	0.000	2.000	487.0
TLLOC	16.381	95.350	1.000	4.000	7225.0
TLOC	20.927	124.8	1.000	4.000	8199.0
TNOS	12.044	68.287	0.000	3.000	4923.0
Halstead					
HOR_D	6.307	4.706	0.000	5.000	37.000
HOR_T	49.515	200.2	0.000	10.000	1674.0
HON_D	25.313	82.235	0.000	9.000	989.0
HON_T	57.229	214.5	0.000	13.000	2415.0
HLEN	106.7	414.2	0.000	23.000	3586.0
HVOC	31.620	84.041	0.000	14.000	1010.0
HDIFF	6.458	7.238	0.000	3.900	98.118
HVOL	787.0	3857.8	0.000	85.837	35,789

Table 1.1: Ferenc Dataset Summary Statistics (continued)

Metric	Mean	Std Dev	Min	Median	Max
HEFF	13,954	74,490	0.000	342.1	917,608
HBUGS	0.262	1.286	0.000	0.029	11.930
HTIME	775.2	4138.3	0.000	19.005	50,978
Comment					
CD	0.042	0.117	0.000	0.000	0.966
CLOC	1.365	7.891	0.000	0.000	259.0
DLOC	0.363	3.040	0.000	0.000	134.0
TCD	0.040	0.106	0.000	0.000	0.966
TCLOC	2.526	20.333	0.000	0.000	736.0
Clone					
CC	0.129	0.331	0.000	0.000	1.000
CCL	0.225	0.887	0.000	0.000	22.000
CCO	11.465	49.696	0.000	0.000	358.0
CI	0.251	1.159	0.000	0.000	48.000
CLC	0.162	0.364	0.000	0.000	1.000
CLLC	0.000	0.000	0.000	0.000	0.000

Note: Std Dev is the standard deviation. Values are rounded to three decimal places where applicable.

1.2 Viszkok Dataset Summary Statistic Table

Observations: 8,038 **Features:** 58
Vulnerable: 960 **Non-vulnerable:** 7,078

Table 1.2: Viszkok Dataset Summary Statistics

Metric	Mean	Std Dev	Min	Median	Max
Complexity					
CYCL	2.572	3.412	1.000	1.000	121.0
CYCL_DENS	67.985	50.057	0.000	53.846	900.0
McCC	2.876	4.662	1.000	1.000	137.0

Table 1.2: Viszkok Dataset Summary Statistics (continued)

Metric	Mean	Std Dev	Min	Median	Max
NL	0.680	1.099	0.000	0.000	14.000
NLE	0.631	0.977	0.000	0.000	9.000
NUMPAR	0.986	1.098	0.000	1.000	13.000
PARAMS	0.967	1.077	0.000	1.000	13.000
Size / LOC					
LDC	9.574	107.7	0.000	0.000	8176.0
LLDC	7.630	91.162	0.000	0.000	7207.0
LLOC	7.223	18.647	1.000	3.000	1071.0
LOC	9.315	29.467	1.000	3.000	1244.0
NOS	4.691	10.003	0.000	2.000	487.0
TLLOC	14.026	100.4	1.000	3.000	7225.0
TLOC	17.725	124.9	1.000	3.000	8199.0
TNOS	11.169	74.399	0.000	2.000	4923.0
Halstead					
HBUGS	0.355	1.566	0.000	0.028	11.930
HDIFF	6.340	7.058	0.000	3.750	98.118
HEFF	18,650	89,938	0.000	311.3	917,608
HLEN	136.9	502.8	0.000	22.000	3586.0
HON_D	31.919	99.636	0.000	9.000	989.0
HON_T	72.900	260.1	0.000	13.000	2415.0
HOR_D	6.297	4.692	0.000	5.000	31.000
HOR_T	64.009	243.2	0.000	10.000	1674.0
HTIME	1036.1	4996.5	0.000	17.293	50,978
HVOC	38.216	101.6	0.000	13.000	1010.0
HVOL	1063.5	4697.4	0.000	84.000	35,789
Comment					
CD	0.033	0.100	0.000	0.000	0.917
CLOC	1.020	6.717	0.000	0.000	204.0
DLOC	0.191	1.235	0.000	0.000	32.000
TCD	0.033	0.093	0.000	0.000	0.917
TCLOC	1.813	16.430	0.000	0.000	736.0
Clone					

Table 1.2: Viszkok Dataset Summary Statistics (continued)

Metric	Mean	Std Dev	Min	Median	Max
CC	0.348	0.474	0.000	0.000	1.000
CCL	0.715	3.029	0.000	0.000	224.0
CCO	156.2	390.8	0.000	0.000	3642.0
CI	0.747	3.432	0.000	0.000	254.0
CLC	0.375	0.481	0.000	0.000	1.000
CLLC	0.375	0.481	0.000	0.000	1.000
Process					
AVGNOAL	15.050	111.7	1.000	3.000	8199.0
AVGNODL	0.560	17.458	0.000	0.000	1545.0
AVGNOEMT	3398.2	2261.9	0.556	3094.0	11,606
AVGNOML	0.258	1.826	0.000	0.000	114.2
AVGTBC	10.579	59.520	0.000	0.000	935.0
MNOAL	17.465	120.5	1.000	3.000	8199.0
MNODL	0.717	17.823	0.000	0.000	1550.0
MNOEMT	3486.0	2243.6	1.000	3094.0	11,606
MNOML	0.453	4.151	0.000	0.000	287.0
Change History					
CChurn	18.112	125.2	-16.000	3.000	8199.0
NII	0.367	1.631	0.000	0.000	40.000
NOADD	1.193	1.588	1.000	1.000	97.000
NOCC	0.109	0.665	0.000	0.000	21.000
NOCHG	1.360	2.520	1.000	1.000	137.0
NOContr	1.068	0.373	1.000	1.000	9.000
NODEL	0.108	0.801	0.000	0.000	46.000
NOI	0.373	1.019	0.000	0.000	18.000
NOMOD	0.249	1.946	0.000	0.000	102.0
SOADD	19.330	138.0	1.000	3.000	8199.0
SODEL	1.217	35.587	0.000	0.000	3090.0
SOMOD	0.963	11.950	0.000	0.000	675.0

Note: Std Dev is the standard deviation. Values are rounded to three decimal places where applicable.